Lecture 11: Machine Learning

Tree-Based Machine Learning Methods

James Sears*
AFRE 891/991 FS25
Michigan State University

*Parts of these slides are adapted from **"Prediction and Machine-Learning in Econometrics"** by Ed Rubin, used under **CC BY-NC-SA 4.0**.

Table of Contents

Part 3: Tree-Based Methods

1. **Decision Trees**

2. **Random Forests**

3. **Machine Learning for Causal Treatment Effect Estimation**

Prologue

Packages we'll use today:

```
pacman::p_load(grf, fixest, janitor, magrittr, parsnip, ranger, rpart,
rpart.plot, rsample, scales, tidymodels, tidyverse, vip)
```

Prologue

Throughout our econometric training, we've become experts in predicting an outcome by writing down a model with a **specific functional form** for the relationship between $\mathbf{y}$ and $\mathbf{X}$.

Decision Trees (or classification and regression trees, CART) are a supervised learning method that offers an alternative, **data-driven** way of generating predictions by partitioning the covariate space into groups and using the grouping to generate predictions.

After working through decision trees, we'll shift our focus to **forest methods** that combine many, many trees to overcome drawbacks of individual trees and yield predictions of alternate objects - including heterogeneous treatment effects.

🌲 Decision Trees 🌲

Decision Trees: Punchline

Goal

- Split the *predictor space* (our $\mathbf{X}$) into regions (partitions)
- Predict the most-common value within a region

Attributes of Decision Trees

1. Work for **both classification and regression**
2. Are inherently **nonlinear**
3. Are relatively **simple** and **interpretable**
4. Often **underperform** relative to competing methods
5. easily extend to **very competitive ensemble methods** (forests with *many* trees) 🌲

🌲 Though the ensembles will be much less interpretable.

Growing Decision Trees

1. Divide the predictor space into J regions (using predictors $\mathbf{x}_1, \ldots, \mathbf{x}_p$)

- **Regression Trees:** Choose the regions to **minimize RSS** across all J regions

$$\sum_{j=1}^{J} \left(y_i - \hat{y}_{R_j} \right)^2$$

Growing Decision Trees: Steps

1. Divide the predictor space into J regions (using predictors $\mathbf{x}_1, \ldots, \mathbf{x}_p$)

- **Classification Trees:** Choose the regions to **minimize Gini index** or **Entropy**

$$G = \sum_{k=1}^{K} \hat{p}_{mk} (1 - \hat{p}_{mk}) \qquad Entropy = -\sum_{k=1}^{K} \hat{p}_{mk} \log(\hat{p}_{mk})$$

- Keep splitting until a specified threshold is reached (e.g. at most 5 observations per region)

Classification Tree

Q: Why are we using the Gini index or entropy (*vs.* error rate)?

A: The error rate isn't sufficiently sensitive to grow good trees.
The Gini index and entropy tell us about the **composition** of the leaf.

Consider two different leaves in a three-level classification.

Leaf 1

- A: 51, B: 49, C: 00
- **Error rate:** 49%
- **Gini index:** 0.4998
- **Entropy:** 0.6929

Leaf 2

- A: 51, B: 25, C: 24
- **Error rate:** 49%
- **Gini index:** 0.6198
- **Entropy:** 1.0325

The **Gini index** and **entropy** tell us about the distribution.

Growing Decision Trees: Steps

2. Make predictions using the regions' mean outcome.

For region R_j predict $\hat{y}_{R_j}$ where

$$\hat{y}_{R_j} = \frac{1}{n_j} \sum_{i \in R_j} y$$

Growing Decision Trees

Let's visualize this by growing a classification tree for predicting credit card defaults based on income and card balances.

Growing Decision Trees

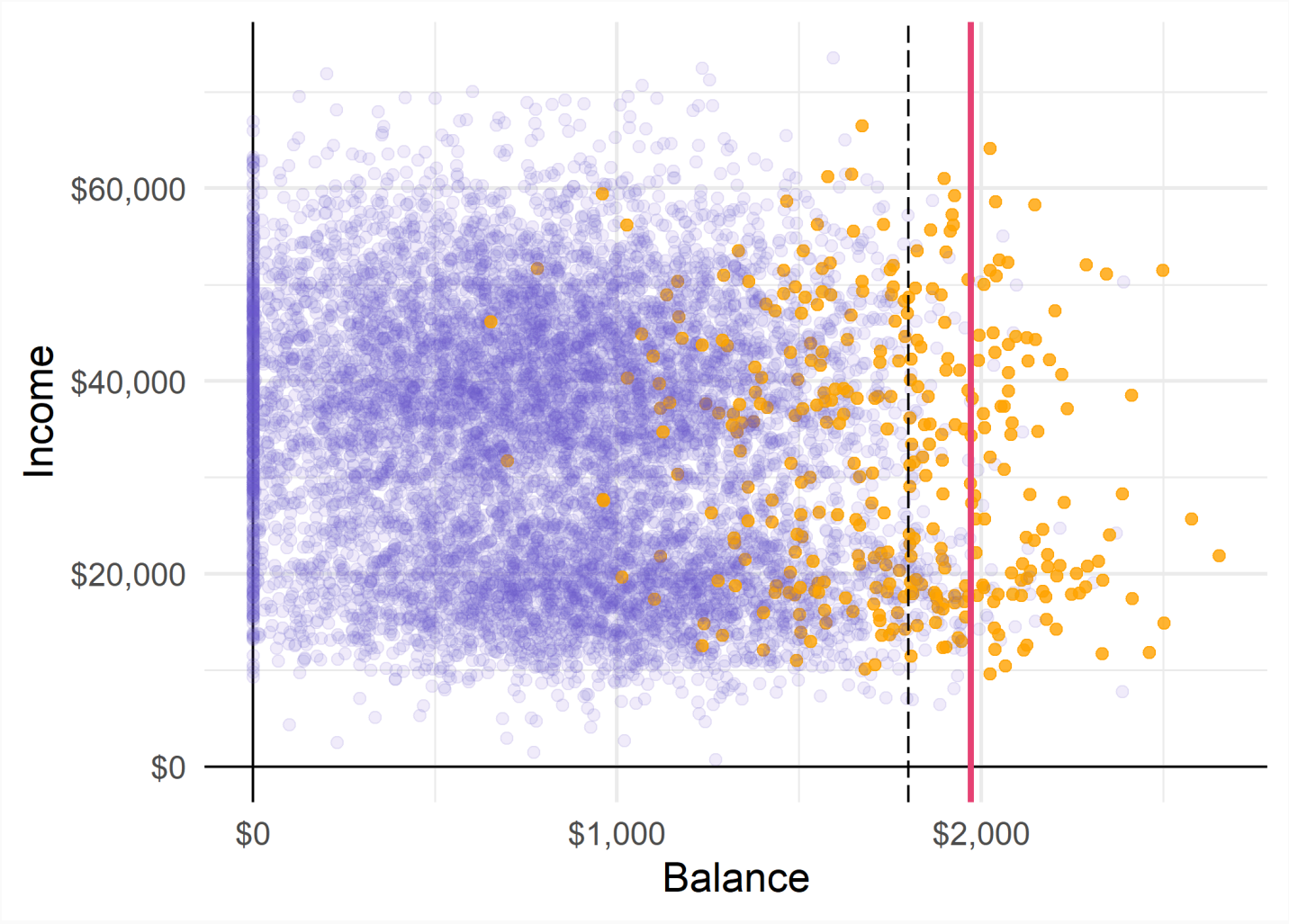
Consider our two-dimensional covariate space:

Growing Decision Trees

The **first partition** splits balance at $1,800.

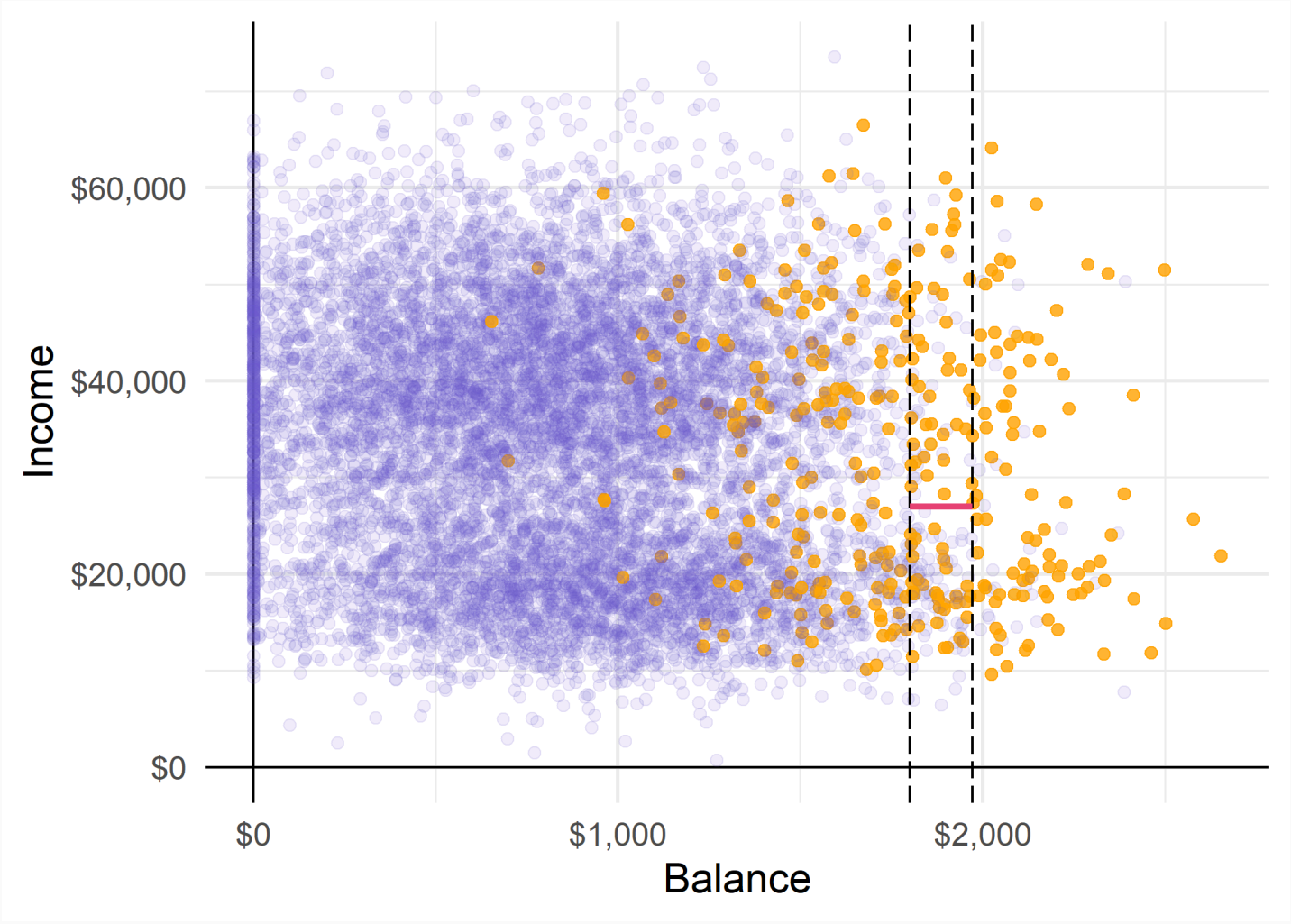
Growing Decision Trees

The **second partition** splits balance at \$1,972, (*conditional on bal. > \$1,800*).



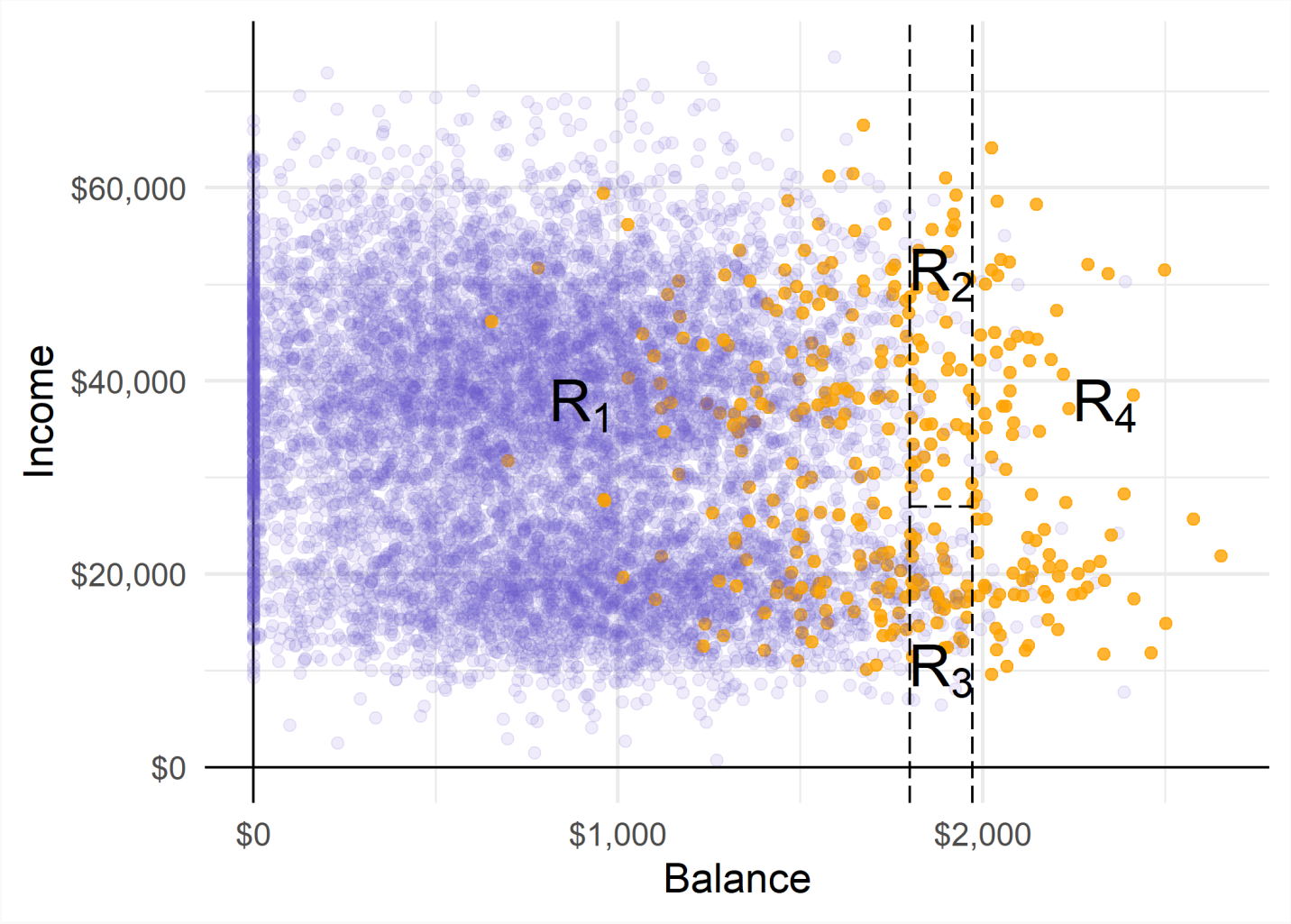
Growing Decision Trees

The **third partition** splits income at $27K for bal. between $1,800 and $1,972.



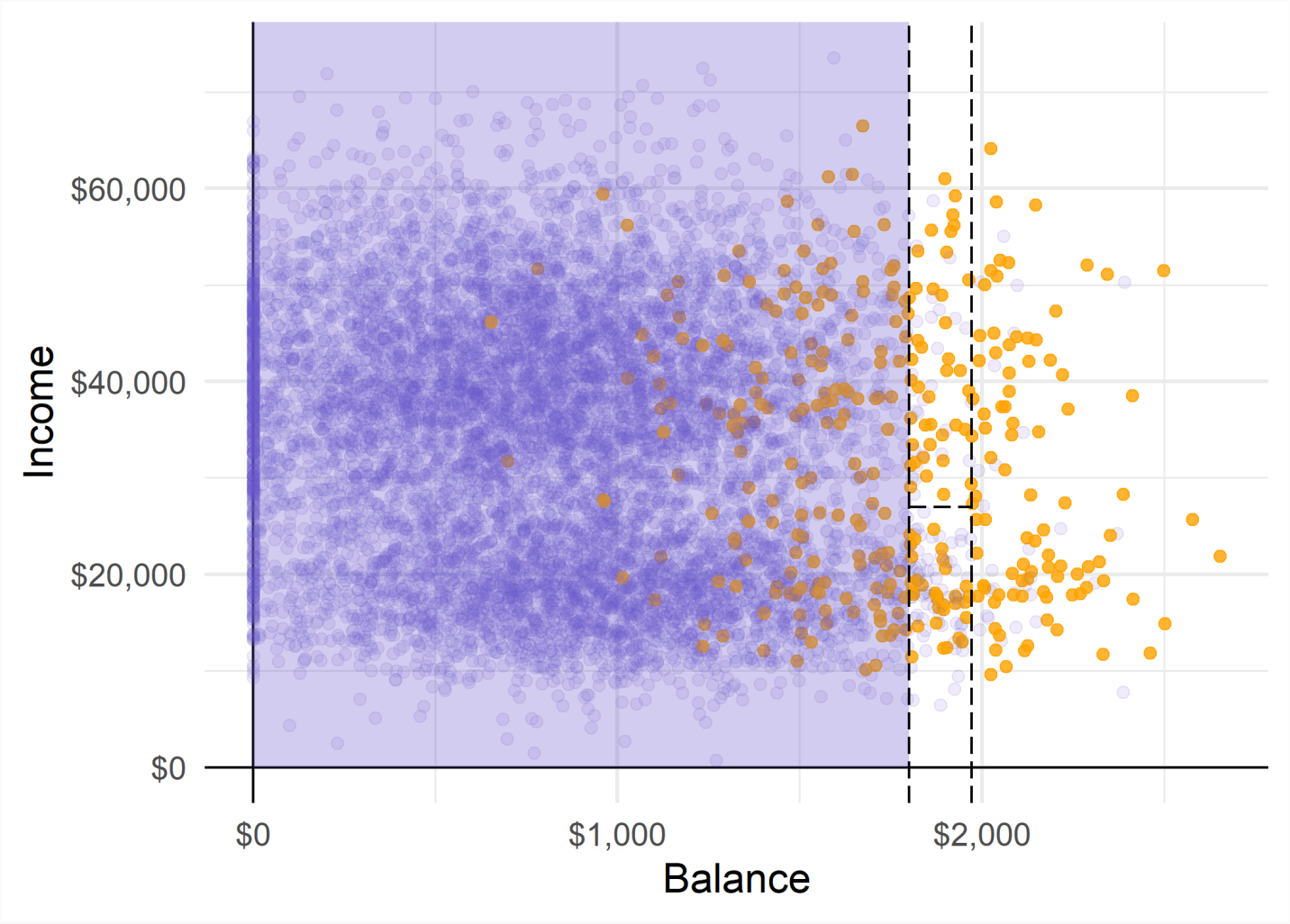
Growing Decision Trees

These three partitions give us four **regions**...



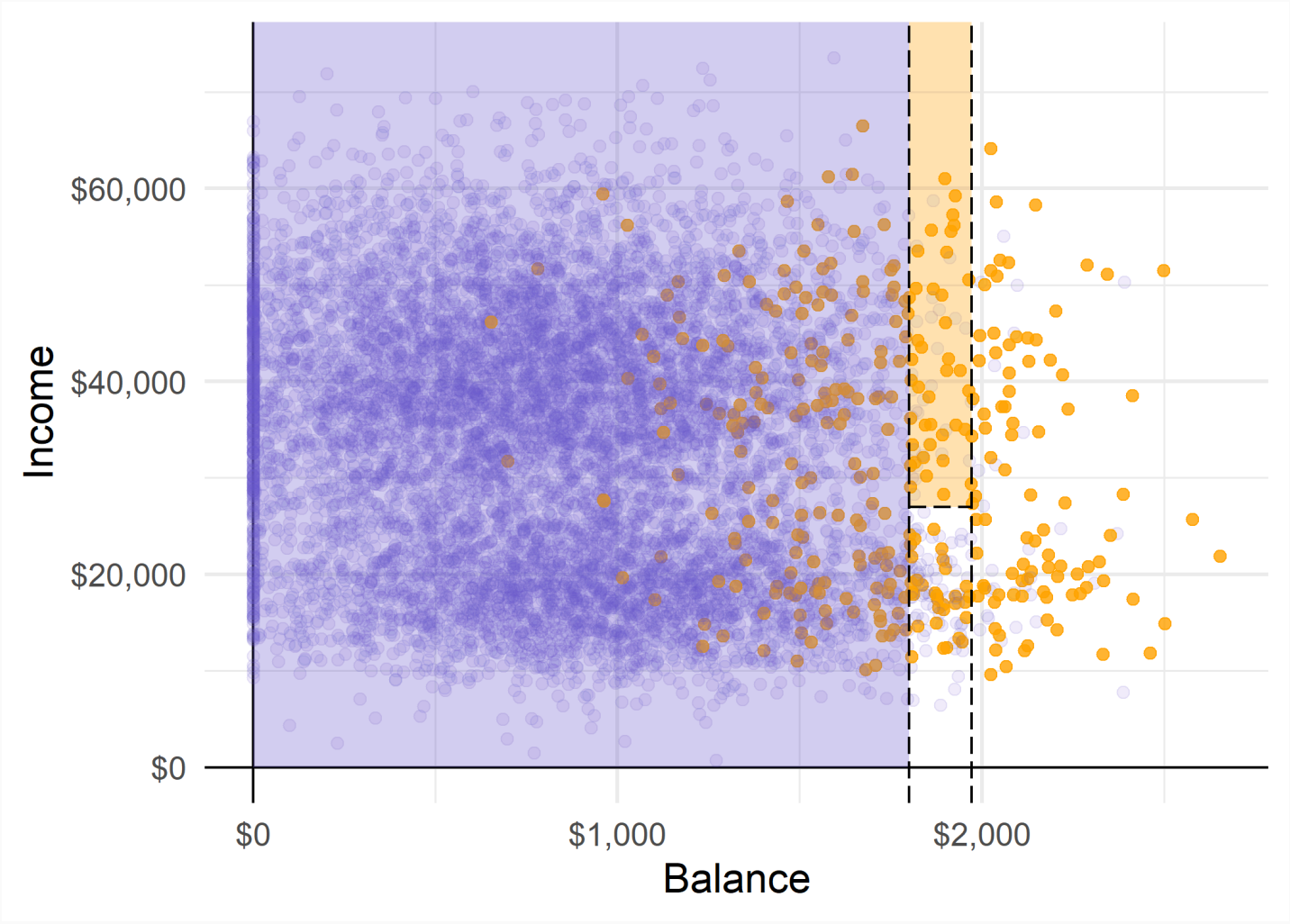
Growing Decision Trees

Predictions cover each region (e.g. using the region's **most common class**).



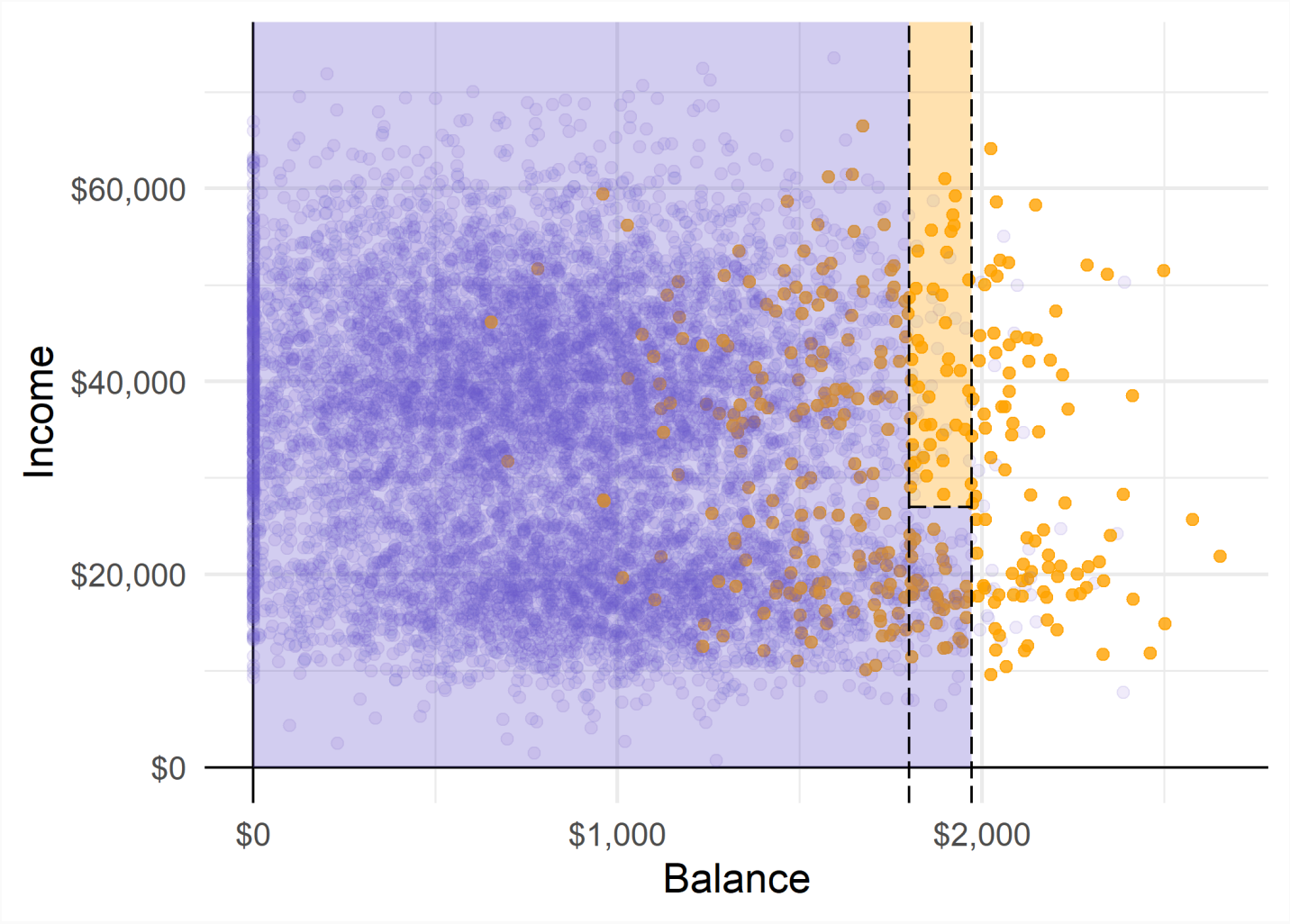
Growing Decision Trees

Predictions cover each region (e.g. using the region's **most common class**).



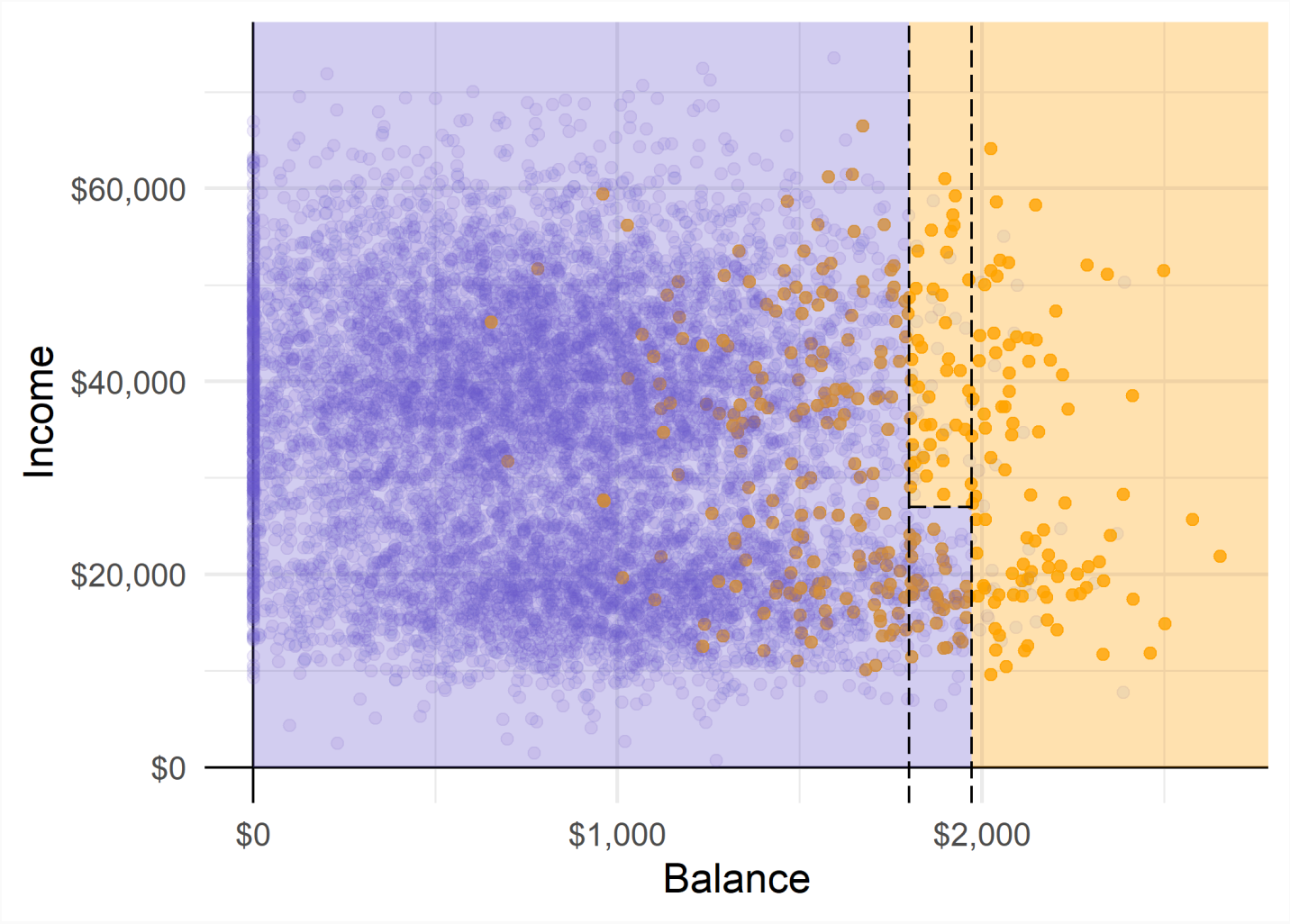
Growing Decision Trees

Predictions cover each region (e.g. using the region's **most common class**).



Growing Decision Trees

Predictions cover each region (e.g. using the region's **most common class**).



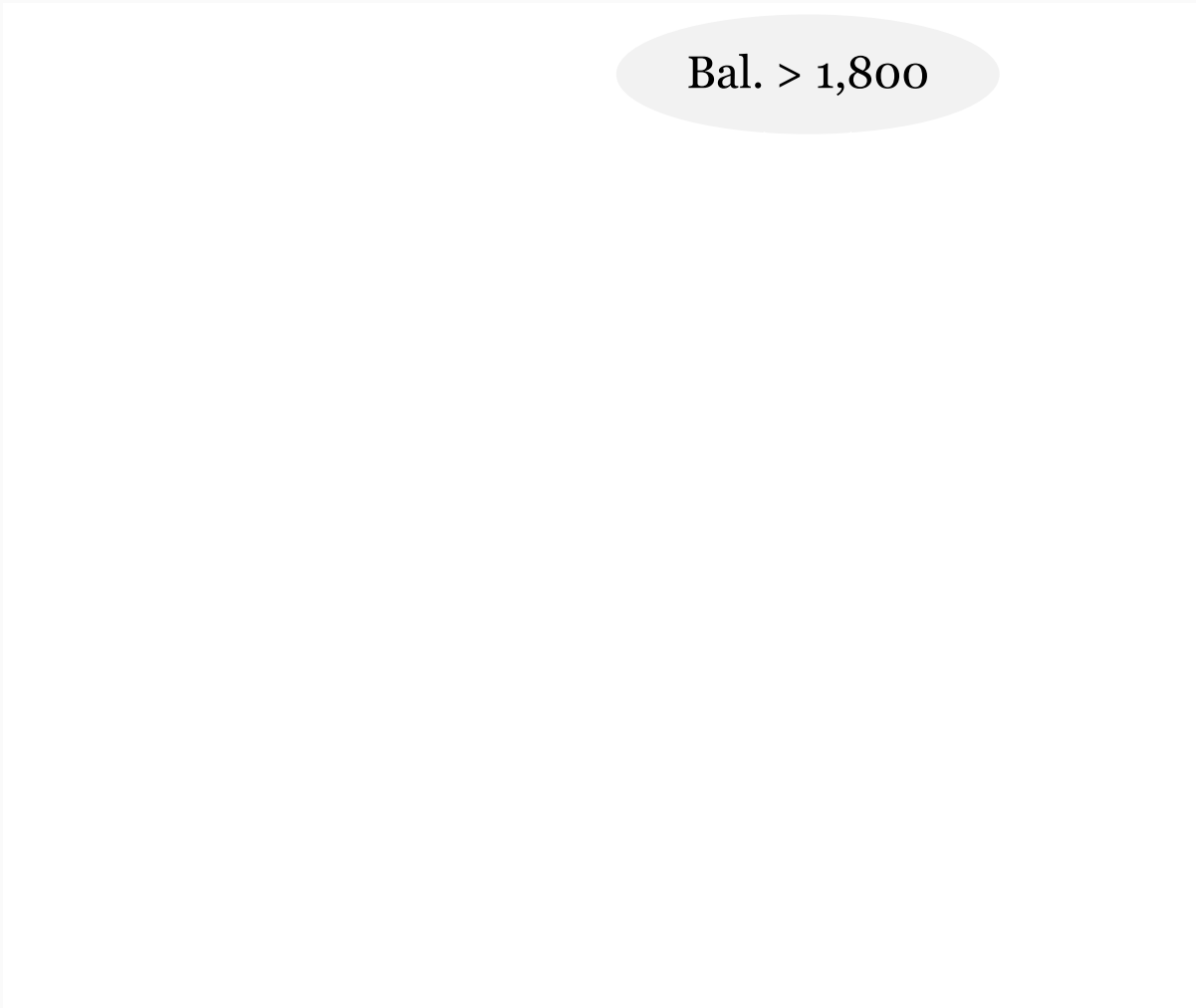
Visualizing the Decision Tree

Q: Why do we call it a tree?

A: Because we can easily represent this partitioning process in the form of a **decision tree!**

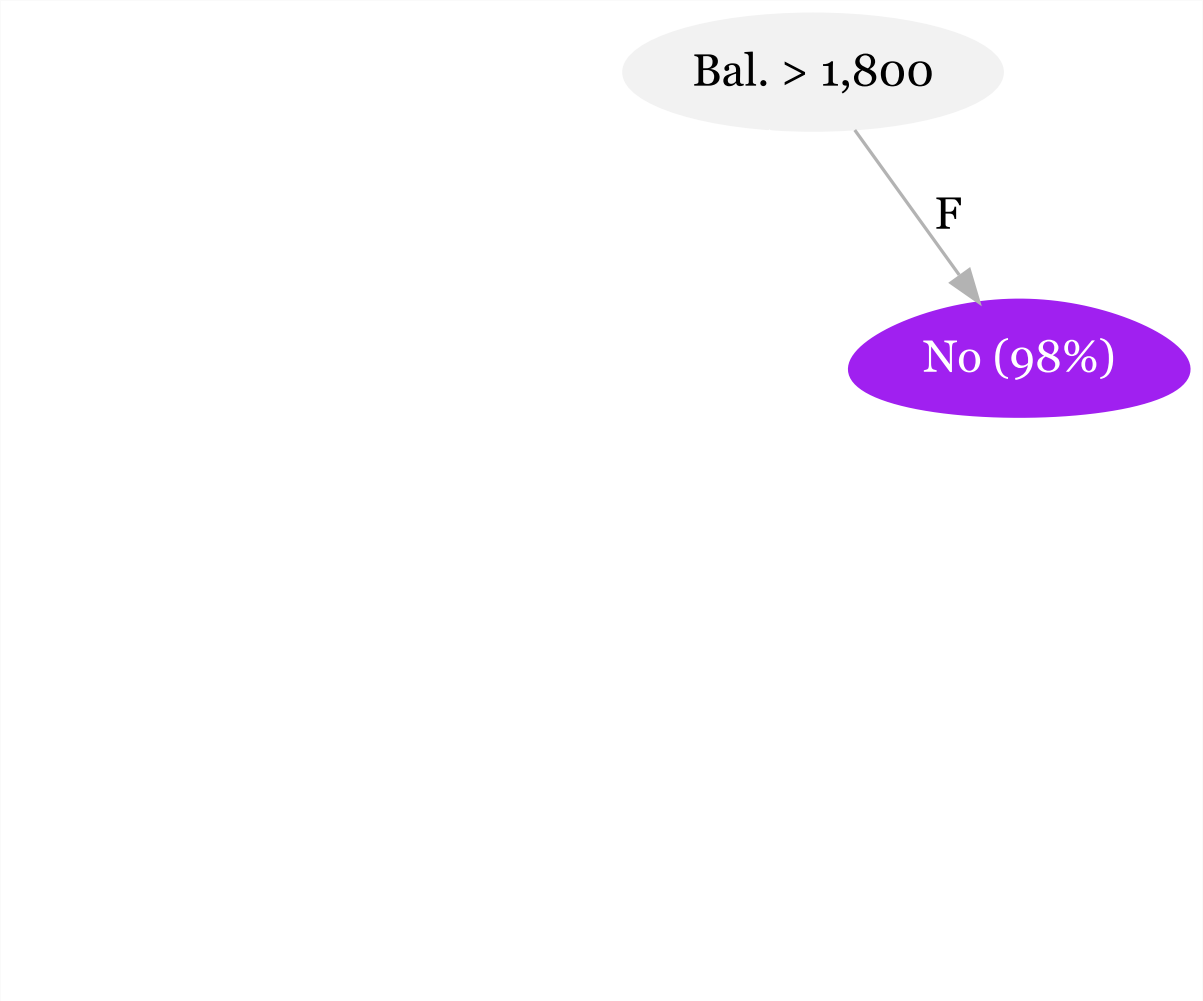
Visualizing the Decision Tree

Our first split occurs at a Balance of $1800:



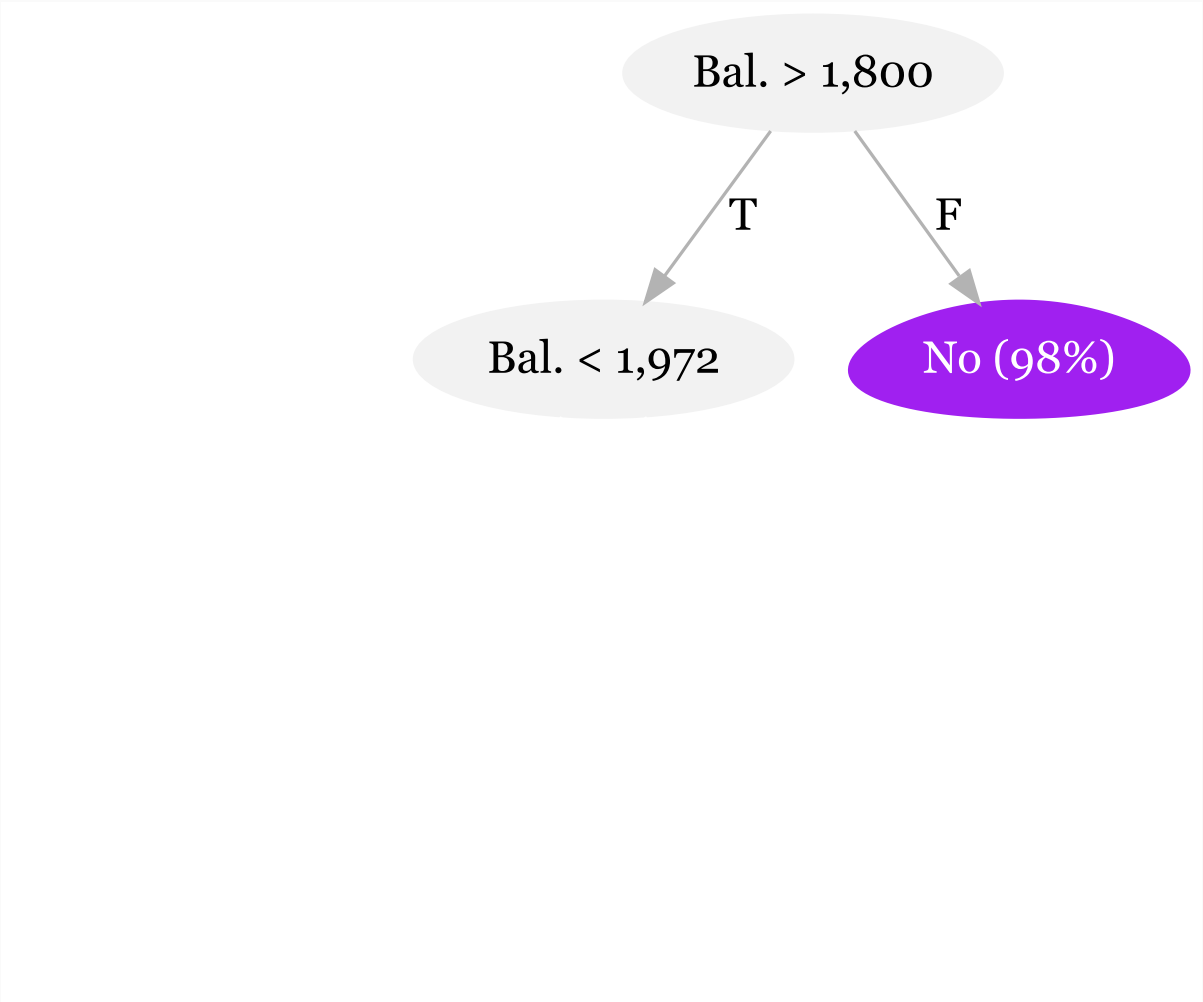
Visualizing the Decision Tree

If Balance > 1800 Is False, we get our first region:



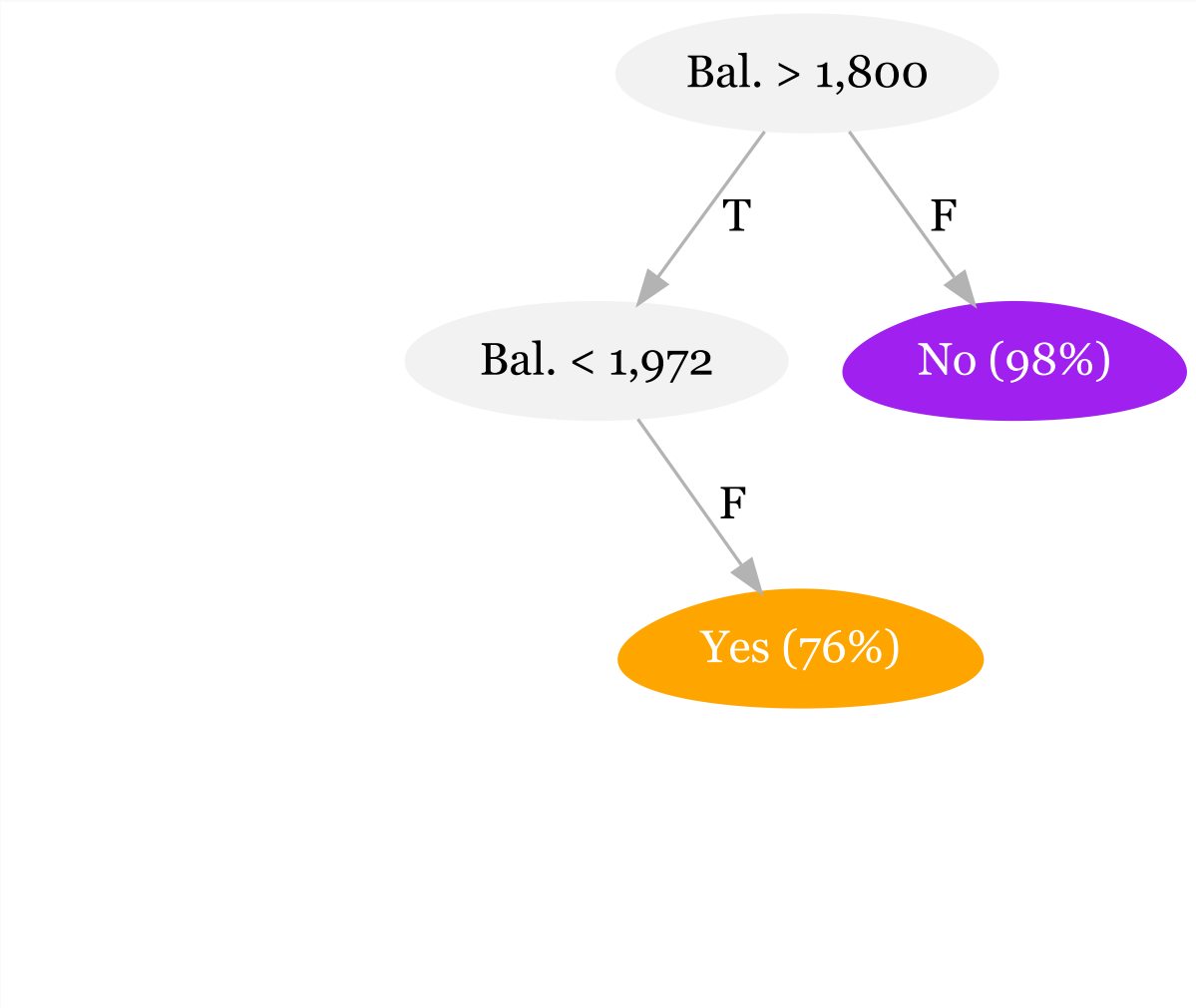
Visualizing the Decision Tree

If Balance < 1800 Is True, we next split on Balance < 1,972



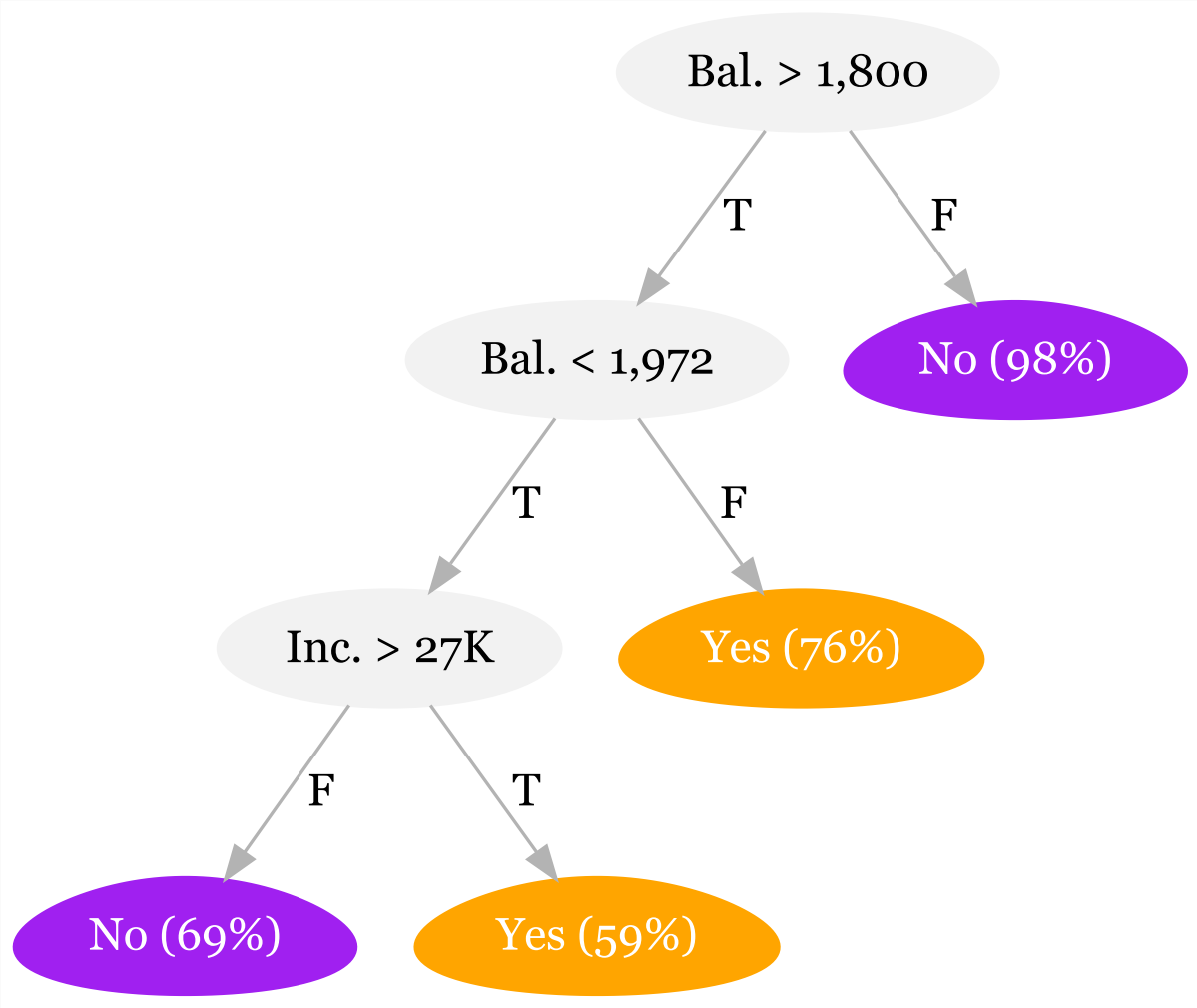
Visualizing the Decision Tree

If False, we get our second terminal node (76% default)



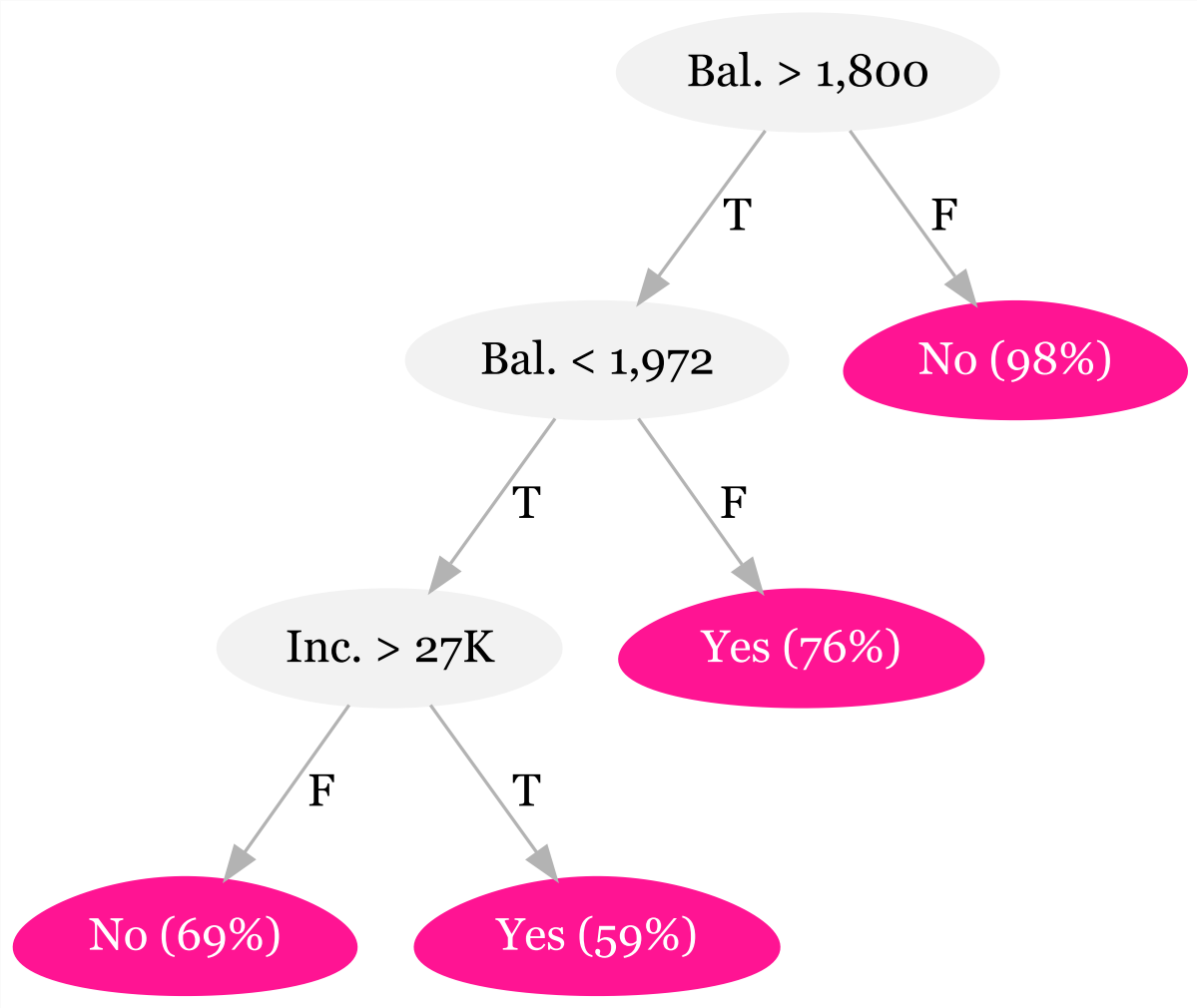
Visualizing the Decision Tree

If True, we get our final split at Income > 27,000



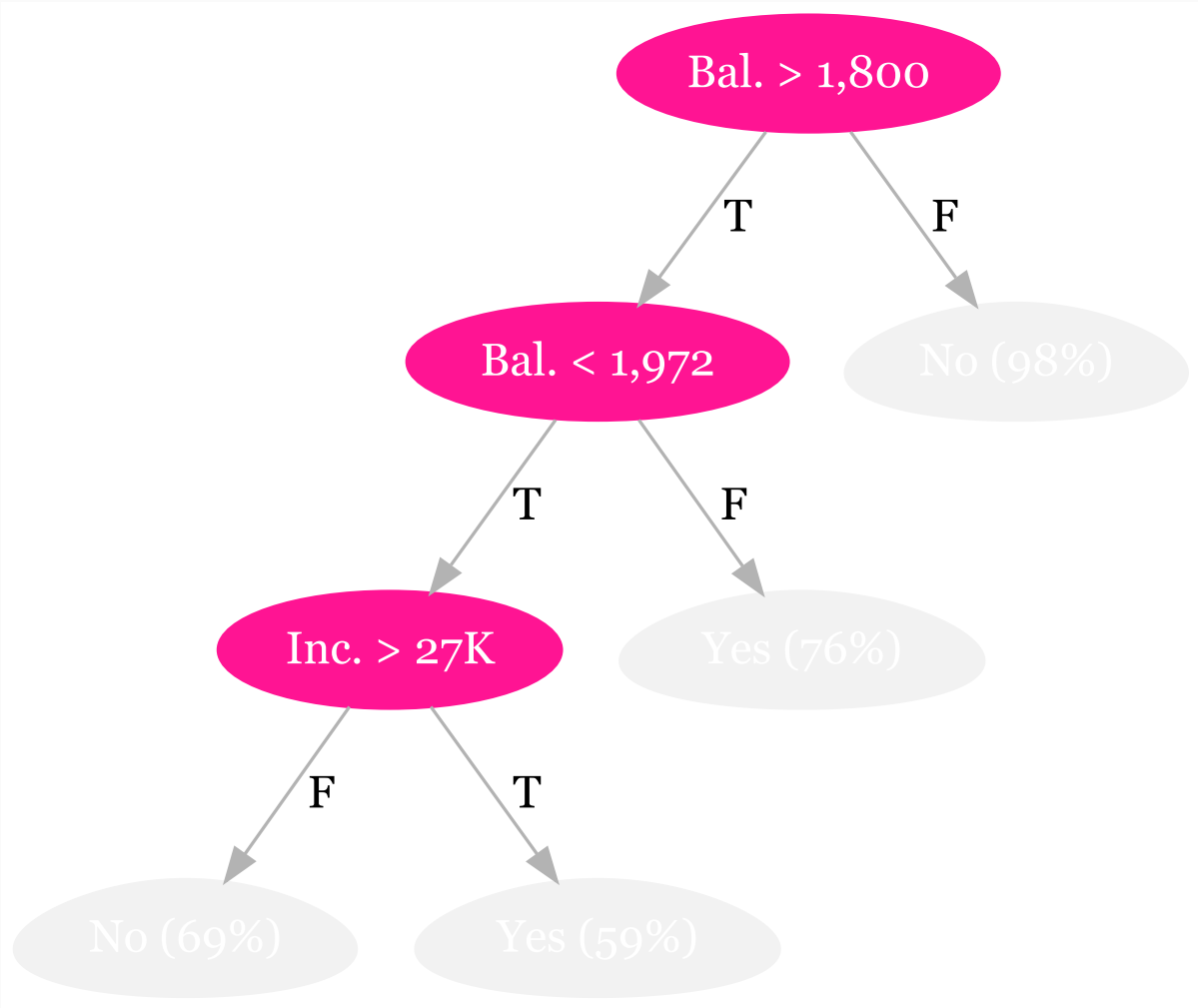
Decision Tree Anatomy

The **regions** correspond to the tree's **terminal nodes** (or **leaves**)



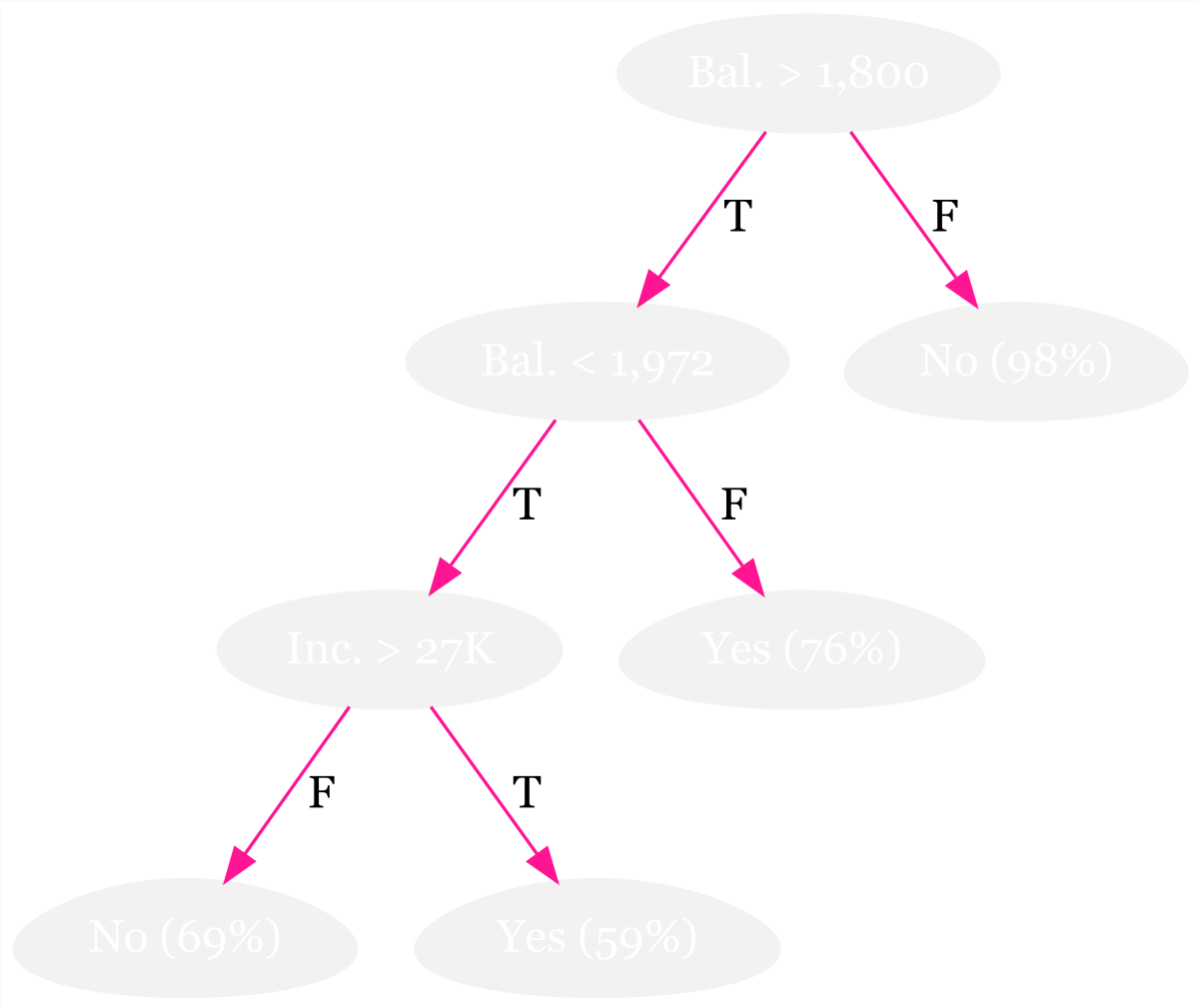
Decision Tree Anatomy

The graph's **separating lines** correspond to the tree's **internal nodes**



Decision Tree Anatomy

The segments connecting the nodes are the tree's **branches**



Growing The Tree

Problem: Examining every possible partition (every cutoff s of every predictor $\mathbf{x}_j$) is **computationally infeasible.**

Solution: a top-down, *greedy* algorithm named **recursive binary splitting**

- **Recursive:** starts with the "best" split, then find the next "best" split, etc.
- **Binary:** each split creates only two branches: "yes" and "no"
- **Greedy:** each step makes the "best" split without consideration for the overall process

Splitting Example

Consider the dataset

i	y	x1	x2
1	0	1	4
2	8	3	2
3	6	5	6

With just three observations, each variable only has two actual splits. 🌲

🌲 You can think about cutoffs as the ways we divide observations into two groups.

Splitting Example

One possible split: x_1 at 2, which yields (1) $x_1 < 2$ *vs.* (2) $x_1 \geq 2$

i	y	x1	x2
1	0	1	4
2	8	3	2
3	6	5	6

Splitting Example

One possible split: x_1 at 2, which yields (1) $x_1 < 2$ *vs.* (2) $x_1 \geq 2$

i	pred	y	x1	x2
1	**0**	0	1	4
2	**7**	8	3	2
3	**7**	6	5	6

This split yields an RSS of $0^2 + 1^2 + (-1)^2 = 2$.

Note$_1$ Splitting x_1 at 2 yields the same results as 1.5, 2.5—anything in (1, 3).

Splitting Example

An alternative split: x_1 at 4, which yields (1) $x_1 < 4$ *vs.* (2) $x_1 \geq 4$

i	pred	y	x1	x2
1	**4**	0	1	4
2	**4**	8	3	2
3	**6**	6	5	6

This split yields an RSS of $(-4)^2 + 4^2 + 0^2 = 32$.

Previous: Splitting x_1 at 4 yielded RSS = 2. *(Much better)*

Splitting Example

Another split: x_2 at 3, which yields (1) $x_1 < 3$ *vs.* (2) $x_1 \geq 3$

i	pred	y	x1	x2
1	**3**	0	1	4
2	**8**	8	3	2
3	**3**	6	5	6

This split yields an RSS of $(-3)^2 + 0^2 + 3^2 = 18$.

Splitting Example

Final split: x_2 at 5, which yields (1) $x_1 < 5$ *vs.* (2) $x_1 \geq 5$

i	pred	y	x1	x2
1	**4**	0	1	4
2	**4**	8	3	2
3	**6**	6	5	6

This split yields an RSS of $(-4)^2 + 4^2 + 0^2 = 32$.

Splitting Example

Across our four possible splits (two variables each with two splits)

- x_1 with a cutoff of 2: RSS = 2
- x_1 with a cutoff of 4: RSS = 32
- x_2 with a cutoff of 3: RSS = 18
- x_2 with a cutoff of 5: RSS = 32

our split of x_1 at 2 generates the lowest RSS.

Splitting

Note: Categorical predictors work in exactly the same way.
We want to try all possible combinations of the categories.

Ex: For a four-level categorical predictor (levels: A, B, C, D)

- Split 1: A|B|C *vs.* D
- Split 2: A|B|D *vs.* C
- Split 3: A|C|D *vs.* B
- Split 4: B|C|D *vs.* A
- Split 5: A|B *vs.* C|D
- Split 6: A|C *vs.* B|D
- Split 7: A|D *vs.* B|C

we would need to try 7 possible splits.

Splitting

Once we make our a split, we then continue splitting, conditional on the regions from our previous splits.

So if our first split creates R_1 and R_2, then our next split searches the predictor space only in R_1 or R_2. 🌲

The tree continue to grow until it hits some specified threshold, *e.g.*, at most 5 observations in each leaf.

🌲 We are no longer searching the full space—it is conditional on the previous splits.

Too many Splits?

One can have **too many splits.**

Q: Why?

A: "More splits" means

1. more flexibility (think about the bias-variance tradeoff/overfitting)
2. less interpretability (one of the selling points for trees)

Q: So what can we do?

A: Prune your trees!

Pruning

The idea: Some regions may increase **variance** more than they reduce **bias**.

- By removing these regions, we gain in test MSE.

Candidates for trimming: Regions that do not reduce RSS very much.

Updated strategy: Grow big trees T_0 and then trim T_0 to an optimal subtree.

Updated problem: Considering all possible subtrees can get expensive.

Pruning

Updated solution: add a penalty for complexity with **cost-complexity pruning**

- Pay a penalty to become more complex with a greater number of regions $|T|$

For regression trees[1]:

$$\sum_{m=1}^{|T|} \sum_{i:x \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha |T|$$

For any value of $\alpha (\geq 0)$, we get a subtree $T \subset T_0$.

We choose α via cross validation.

1 For classification trees we instead penalize the error rate

Decision Trees vs. Linear Models

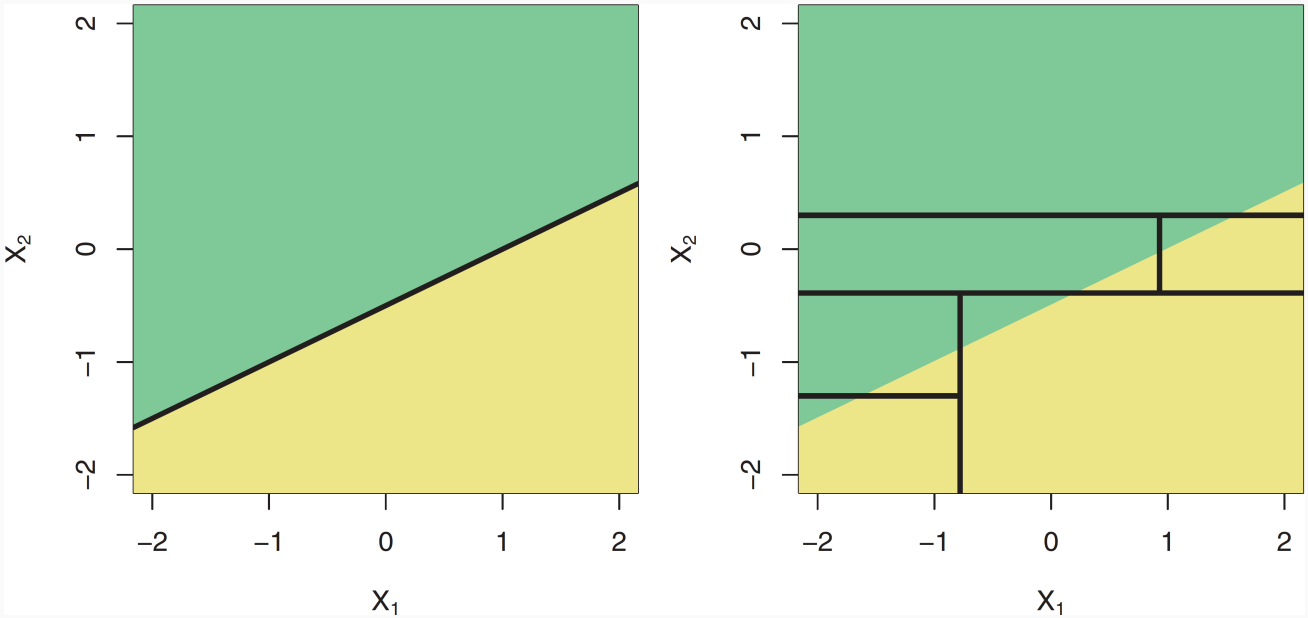
Q How do trees compare to linear models?

A It depends how linear the true boundary is.

Linear Boundary

Linear models recreate the line

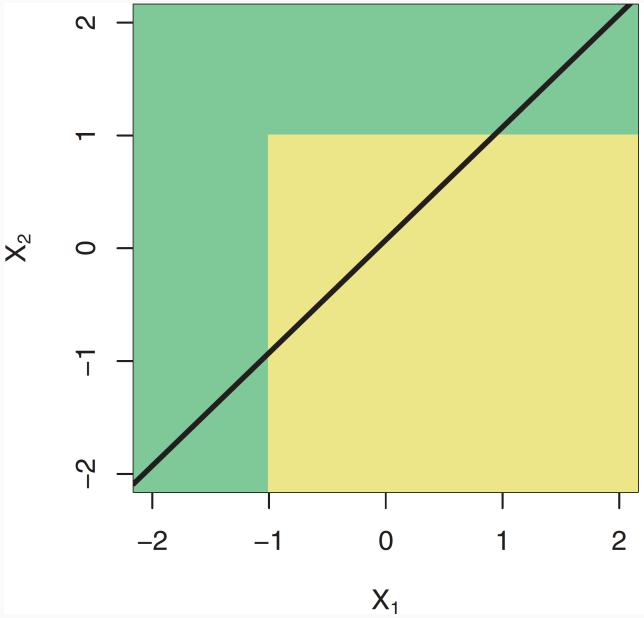
trees struggle at replicating linearity



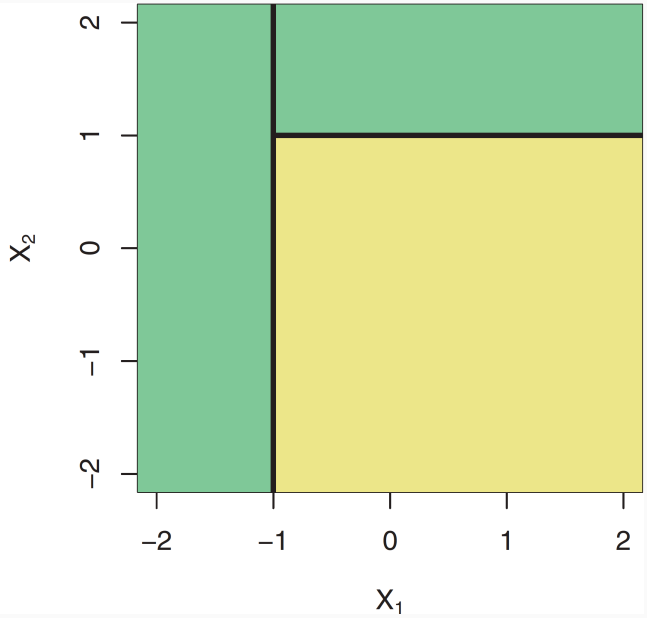
Source: ISL, p. 315

Nonlinear Boundary

Linear models struggle with nonlinear boundaries



trees easily replicate these relationships



Source: ISL, p. 315

Strengths and weaknesses

As with any method, decision trees have **tradeoffs**.

Strengths	Weaknesses
+ Easily explained/interpretted	- Outperformed by other methods
+ Include several graphical options	- Struggle with linearity
+ Mirror human decision making?	- Can be very "non-robust" - small data changes can cause huge changes in our tree
+ Handle num. or cat. on LHS/RHS	

Next: Create ensembles of trees 🌲 to strengthen these weaknesses. 🌴 with...

🌲 Forests! 🌴 Which will also weaken some of the strengths.


Random Forests

Random Forests

The gist: combine many individual trees into a single **forest** via **bootstrap aggregation (bagging)**

Bagging:

1. Create B bootstrapped samples
2. Train an estimator (tree) $\hat{f}^b(x)$ on each of the B samples
3. Aggregate across your B bootstrapped models:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x)$$

This aggregated model $\hat{f}_{\text{bag}}(x)$ is your final model.

Bagging

When we apply bagging to decision trees,

- We typically **grow the trees deep and do not prune**
- For **regression**, we **average** across the B trees' regions
- For **classification**, we have more options—but often take **plurality**

Individual (unpruned) trees will be very **flexible** and **noisy**,
but their **aggregate** will be quite **stable**.

Out-of-Bag Error

Bagging also offers a convenient method for evaluating performance.

For any bootstrapped sample, we omit ~n/3 observations.

Out-of-bag (OOB) error estimation estimates the test error rate using observations **randomly omitted** from each bootstrapped sample.

For each observation i:

1. Find all samples S_i in which i was omitted from training.
2. Aggregate the $|S_i|$ predictions $\hat{f}^b(x_i)$, *e.g.*, using their mean or mode
3. Calculate the error, *e.g.*, $y_i - \hat{f}_{i,\text{OOB},i}(x_i)$

Out-of-Bag Error

When B is big enough, the OOB error rate will be very close to LOOCV.

Q: Why use OOB error rate?

A: When B and n are large, cross validation—with any number of folds—can become pretty computationally intensive.

Random Forests

Random forests overcome a major shortcoming of bagging: **very correlated trees**

- Since trees consider all predictors at every split of every tree, the trees will often be highly related.

Solution: decorrelate the trees!

- Only consider a **random subset** of m $\left(\approx \sqrt{p}\right)$ predictors when making each split (for each tree)
- This nudges trees away from always using the same variables, increasing variation across trees, and potentially reducing the variance of our estimates
- If our predictors are very correlated, we may want to shrink m

Random Forests

Random forests thus introduce **two dimensions of random variation**

1. The **bootstrapped sample**
2. The m **randomly selected predictors** (for the split)

Everything else about random forests works just as it did with decision trees.

Random Forests

Random Forests offer a lot of improvement over single decision trees from a performance standpoint.

However, they can often be harder to interpret.

Perhaps more limiting to the types of questions we economists are interested in: it's not always a **conditional mean** that we're interested in.

What if we want to harness the benefits of **forest-based ensemble methods** for identifying **any conditional object of interest?**

Machine Learning for Causal Treatment Effect Estimation

Machine Learning for Causal Treatment

The goal of **Random Forests** is to estimate a **conditional mean**,

$$\mu(x_0) = E[Y \mid X = x_0]$$

But what if instead we wanted to estimate... **any** conditional function $\theta(x_0)$?

- Conditional linear regression coefficients
- Conditional causal treatment effects
- Conditional local average treatment effects using instruments
- Quantiles of the conditional distribution of $Y \mid X = x_0$
- Conditional class probabilities

Enter **Athey, Tibshirani, and Wager (2019)**.

Generalized Random Forests

Generalized Random Forests by **Athey, Tibshirani, and Wager (2019)** extends the ensemble methods of Random Forests to essentially any conditional function $\theta(x)$ that can be identified by local moment conditions.

They recast forests as an **adaptive locally weighted estimator**

1. Use the forest to calculate weighted set of neighbors for each observation (sound familiar?)
    - Employ problem-specific splitting rules to maximize the likelihood of capturing heterogeneity in $\theta(x)$
2. Use those neighbors to estimate $\theta(x)$ as the solution to a local moment equation

GRF Setup

More formally:

Suppose you have an IID sample with N observations.

You observe:

- O_i an observable quantity that encodes information relevant to estimating $\theta(\cdot)$
  - For nonparametric regression, this is just our our outcome $O_i = \{Y_i\}$
  - For treatment effect estimation, $O_i = \{Y_i, W_i\}$
  - Generally can contain more info
- Auxiliary covariates Xi

GRF

Goal: Estimate solutions to local estimating equations of the form

$$E[\psi_{\theta(x),\, \nu(x)}(O_i \mid X_i = x)] = 0$$

- $\theta(x)$ the parameter we care about (i.e. conditional mean, treatment effect, regression slope)
- $\nu(x)$ a (optional) nuisance parameter

If conditional mean without noise, then $\psi_{\mu(x)}(Y_i) = Y_i - \mu(x)$

GRF Forest-Based Local Estimation

One approach to estimating functions like $\theta(x)$:

- Find $\hat{\theta}(x), \hat{\nu}(x)$ that solve

$$\sum_{i=1}^{n} \alpha_i \psi_{\hat{\theta}(x), \hat{\nu}(x)}(O_i) = 0$$

- α_i: weights measuring relevance of the i^{th} observation for fitting $\theta(\cdot)$ at x
  - Traditionally obtained using a kernel function (and sometimes a bandwidth)
  - Traditionally sensitive to curse of dimensionality

GRF Forest-Based Local Estimation

GRF Approach: use forests to derive α_i.

1. Grow $b = 1, \ldots, B$ trees
2. Choose weights $\alpha_i(x)$ to capture how frequently the i^{th} training example falls into the same leaf as x.

Weights within a given tree, $\alpha_{bi}(x)$, are calculated as

$$\alpha_{bi}(x) = \frac{I[X_i \in L_b(x)]}{|L_b(x)|}$$

- $L_b(x)$ the set of observations falling within the same leaf as x

Overall weights for observation i are then the average across all trees:

$$\alpha_i(x) = \frac{1}{B} \sum_{b=1}^{B} \alpha_{bi}(x)$$

GRF Weights

For example, we can reframe random forests and our estimate of the conditional mean function as

$$\hat{\mu}(x) = \sum_{i=1}^{n} \frac{1}{B} \sum_{b=1}^{B} Y_i \frac{I[X_i \in L_b(x)]}{|L_b(x)|}$$

Or as the weighted sum of single tree predictions $\hat{\mu}_b(\cdot)$:

$$\hat{\mu}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{\mu}_b(x)$$

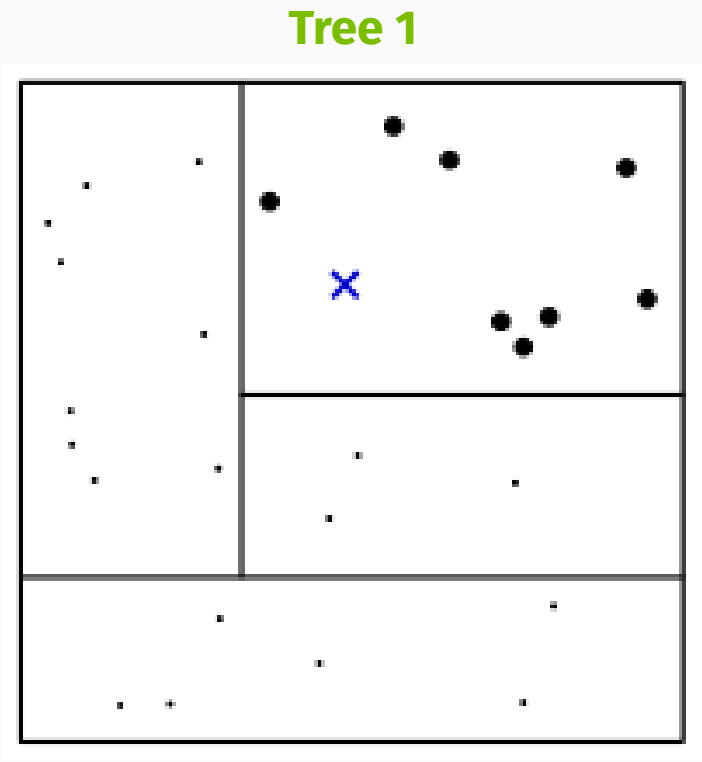
Or more simply, as

$$\hat{\mu}(x) = \sum_{i=1}^{n} Y_i \alpha_i(x)$$

GRF Weights Visually

Let's see how these weights are developed with a simple example: a forest with three trees and two covariates.

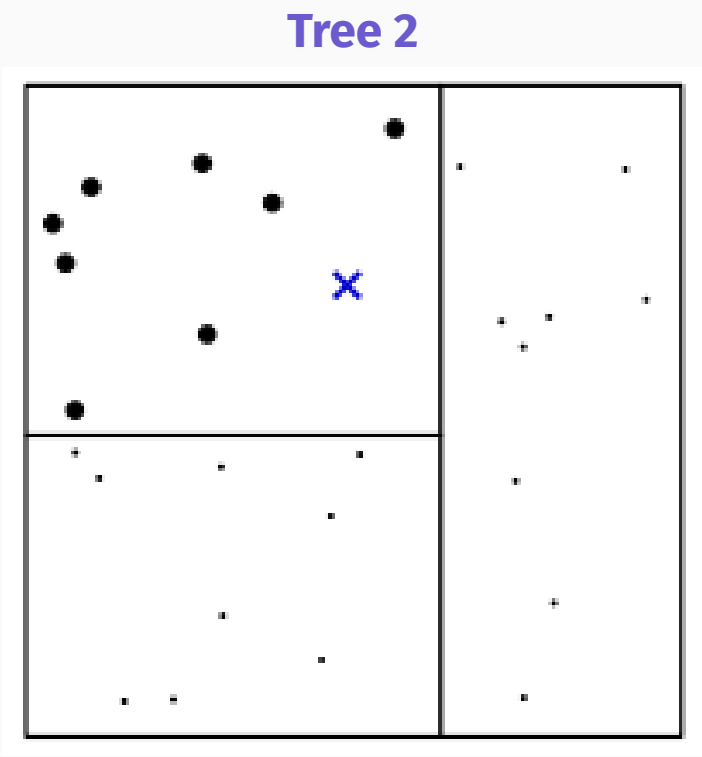
- x the point of interest
  - lines our splits
  - darker dots the points within the same leaf



GRF Weights Visually

Let's see how these weights are developed with a simple example: a forest with three trees and two covariates.

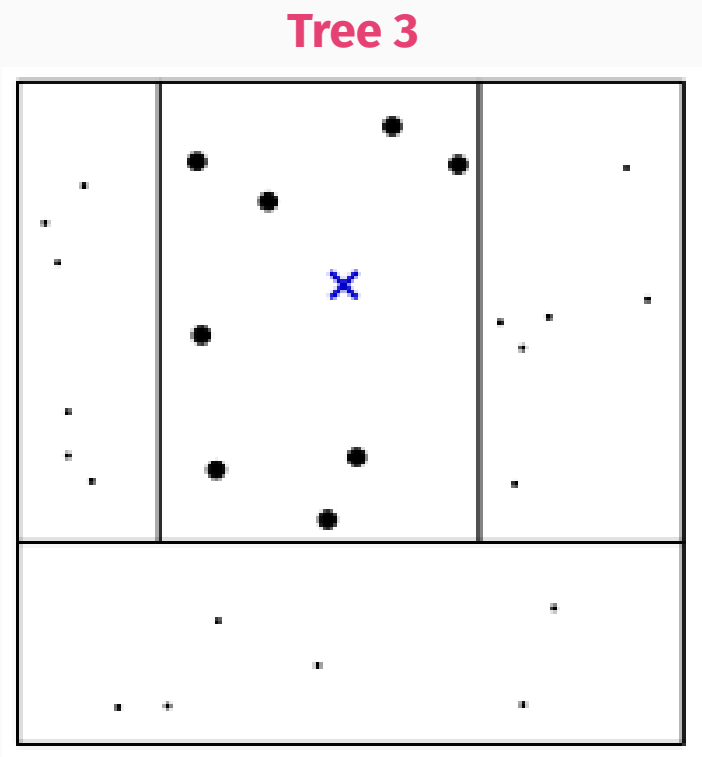
- x the point of interest
  - lines our splits
  - darker dots the points within the same leaf



GRF Weights Visually

Let's see how these weights are developed with a simple example: a forest with three trees and two covariates.

- x the point of interest
  - lines our splits
  - darker dots the points within the same leaf

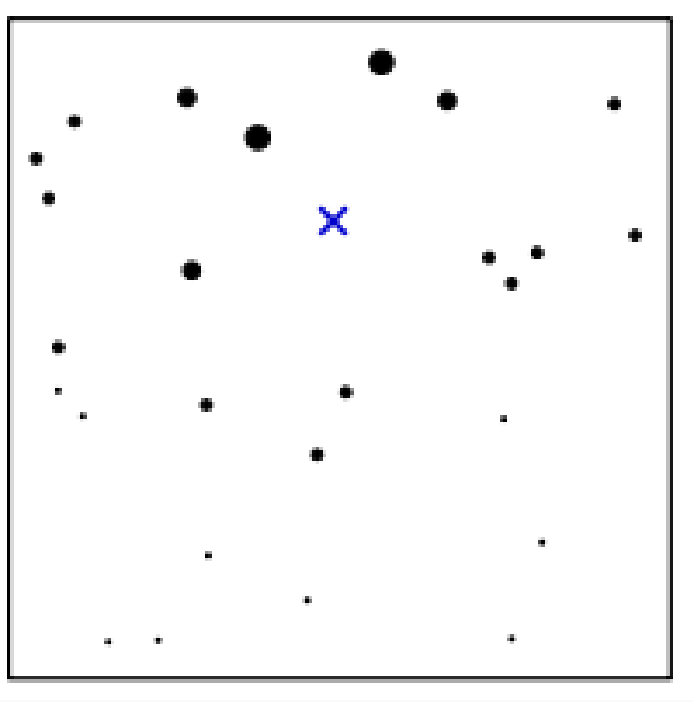


GRF Weights Visually

Let's see how these weights are developed with a simple example: a forest with three trees and two covariates.

Then we average across all the individual tree-based weights to obtain the overall $\alpha_i(x)$.

Forest Weights for x



Honesty

One additional benefit of using GRF: growing **honest** forests

Honesty aims to reduce bias in tree predictions by using **distinct subsamples** for 1. Building the tree, and 2. Making predictions

Classic Random Forest

- Draws a single subsample, use that subsample for both choosing a tree's splits and making predictions in that tree

Honest Forests

- Draw a training sample
- Use part of that sample to choose the tree's splits
- Use the rest of the training sample to make predictions
- Prune away any remaining empty leaves

GRF Functions

Conveniently for us, the **grf** package contains tons of features and **extensive documentation** to estimate a wide range of forests, including...

Causal Treatment Effects

Approach	Forest Function
Causal Treatment Effects	`causal_forest()`
Causal Treatment Effects with right-censored outcomes	`causal_survival_forest()`
Multi-arm/multi-outcome causal treatment effects	`multi_arm_causal_forest()`

GRF Functions

Conveniently for us, the **grf** package contains tons of features and **extensive documentation** to estimate a wide range of forests, including...

Moments of Conditional Distributions

Approach	Forest Function
Conditional Mean	`regression_forest()`
Conditional Mean	`boosted_regression_forest()`
Conditional Mean	`ll_regression_forest()`
Multi-outcome conditional means	`multi_regression_forest()`
Right-censored Survival	`survival_forest()`
Conditional Quantiles	`quantile_forest()`
Conditional Class Probabilities	`probability_forest()`

GRF Functions

Conveniently for us, the **grf** package contains tons of features and **extensive documentation** to estimate a wide range of forests, including...

Regression Outcomes

Approach	Forest Function
Conditional Linear Model	`lm_forest()`
Local Average Treatment Effects (IV)	`instrumental_forest()`

Causal Forests

When $W_i \in \{0, 1\}$ and **unconfoundedness holds**, we can estimate heterogeneous causal treatment effects with **causal forests**.

1. Build Phase: greedily choose splits to maximize the **squared difference in subgroup treatment effects**

$$n_L n_R (\hat{\tau}_L - \hat{\tau})_R)^2$$

- $\hat{\tau}$'s chosen through a centered residual-on-residual regression (Robinson 1988)

Causal Forests

2. Estimate $\tau(x)$, where

$$\tau(x) := lm\left(Y_i - \hat{m}^{-1}(X_i) \sim W_i - \hat{e}^{-1}(X_i),\ \mathbf{weights} = \alpha_i(x)\right)$$

- $\left(Y_i - \hat{m}^{-1}(X_i),\ W_i - \hat{e}^{-1}(X_i)\right)$: orthogonalized outcome/treatment
  - Residual after "regressing out" the main effect of X_i on Y_i (W_i)
  - $\hat{m}^{-1}(X_i), \hat{e}^{-1}(X_i)$ obtained from separate regression forests

Causal Forests Example

Let's load in some data to see how we can run causal forests, using a sample from the **National Study of Learning Mindsets**.

ARTICLE

OPEN

https://doi.org/10.1038/s41586-019-1466-y

A national experiment reveals where a growth mindset improves achievement

David S. Yeager[1*], Paul Hanselman[2*], Gregory M. Walton[3], Jared S. Murray[1], Robert Crosnoe[1], Chandra Muller[1], Elizabeth Tipton[4], Barbara Schneider[5], Chris S. Hulleman[6], Cintia P. Hinojosa[7], David Paunesku[8], Carissa Romero[9], Kate Flint[10], Alice Roberts[10], Jill Trott[10], Ronaldo Iachan[10], Jenny Buontempo[1], Sophia Man Yang[1], Carlos M. Carvalho[1], P. Richard Hahn[11], Maithreyi Gopalan[12], Pratik Mhatre[1], Ronald Ferguson[13], Angela L. Duckworth[14] & Carol S. Dweck[3]

```
nslm ← read.csv("data/NSLM.csv")
```

Causal Forests Example

Taking a look at the data, we have the following variables:

- `schoolid` the ID of randomly selected US public high schools
- `Y`: continuous measure of achievement
- `Z`: treatment status
- `S3`: students' self-reported expectations for future success
- `C1`: student race (categorical)
- `C2`: student gender (categorical)
- `C3`: first-generation status (categorical)
- `XC`: school urbanicity (categorical)
- `X1`: school-level mean of students' fixed mindsets
- `X2`: school achievement level (test scores + college prep)
- `X3`: school racial minority composition (% black, latino, native american)
- `X4`: school poverty (% in familes below FPL)
- `X5`: school size

Causal Forests Example

We want to answer the following research questions:

1. Was a nudge-like mindset intervention designed to instill a "growth mindset" in students effective at improving achievement?

2. Do schools' prior achievement levels (`X2`) and pre-existing mindset norms (`X1`) effect the magnitude of this effect?

Let's find out!

Causal Forests Example

We *could* just run an (interacted) OLS regression...

```
pacman::p_load(fixest, grf, tidyverse)
reg1 ← feols(Y ~ Z + S3 + C1 + C2 + C3 + XC + X1 + X2 + X3 + X4 + X5, data  =
nslm, vcov = "HC1")
reg2 ← feols(Y ~ Z + Z:X2 + Z:X1 + S3 + C1 + C2 + C3 + XC + X1 + X2 + X3 + X4 +
X5, data  = nslm, vcov = "HC1")
etable(reg1, reg2, keep = c("Z", "X2", "X1"))
```

	reg1	reg2
Dependent Var.:	Y	Y
Z	0.2549*** (0.0115)	0.2519*** (0.0115)
X1	-0.0954*** (0.0073)	-0.0819*** (0.0087)
X2	-0.0163. (0.0087)	-0.0160 (0.0100)
Z x X2		-0.0003 (0.0146)

Causal Forests Example

...which gives us an estimate of the mean treatment effect and mean mediating effect of `X1/X2`.

Instead, let's use `causal_forest()` to obtain heterogeneous treatment effects for each school.

Let's first build some data objects we'll use as arguments.

1. Convert `C1` and `XC` to dummies:

- `C1` has 15 levels, so rather than code each one by hand let's use `model.matrix`
  - Argument: formula

```
# expand C1 categorical variable into dummies (with intercept)
C1_exp ← model.matrix(~ factor(nslm$C1) + 0)
```

Causal Forests Example

Let's first build some data objects we'll use as arguments.

Repeat for `XC`:

```
XC_exp ← model.matrix(~ factor(nslm$XC) + 0)
```

Causal Forests Example

Next, build the predictor matrix

```
# first select all covariates except for XC and C1
X ← select(nslm, -schoolid:-Y, -XC, - C1)

# Add in the dummies for XC and C1
Xmat ← cbind(X, C1_exp, XC_exp)
```

And get the outcome and treatment vectors along with the school clusters:

```
Y ← nslm$Y # our outcome
Z ← nslm$Z # our treatment indicator
schoolid ← nslm$schoolid
```

Causal Forests Example

To perform the residual-on-residual regression step, our forest wants predictions for $\hat{Y}$ and $\hat{W}$. We can either

1. Omit them from the forest setup
    - The forest will estimate them for us in a separate regression forest
2. Run the initial regression forests manually and supply the predictions

Let's do the latter, but first an important consideration...

Causal Forests Example

As with random forests, by default GRF methods assume **independent observations**.

Here, we know that treatment assignment occurred at the **school level**, so students' treatment is dependent on which school that they're at.

The good news is, we can account for this **clustering** within our forest to draw our random units at the school level.

- `clusters` the clustering variable (here our `schoolid` factor)
- `equalize.cluster.weights = TRUE` ensures we draw the same fraction frmo each cluster

Causal Forests Example

Running the initial regression forests for our $\hat{Y}$ and $\hat{W}$ predictions:

```
set.seed(1)
Y_forest ← regression_forest(X = Xmat, Y = Y,
                             clusters = schoolid,
                             equalize.cluster.weights = TRUE)
Y_hat ← predict(Y_forest)$predictions

W_forest ← regression_forest(X = Xmat, Y = Z, clusters = schoolid,
equalize.cluster.weights = TRUE)
W_hat ← predict(W_forest)$predictions
```

Causal Forests Example

Setting up the causal forest:

- `X`: the covariate matrix

```
# Define the random forest
cf ← causal_forest(X = Xmat,
                   Y = Y,
                   Y.hat = Y_hat,
                   W = Z,
                   W.hat = W_hat,
                   clusters = schoolid,
                   equalize.cluster.weights
= TRUE,
                   num.trees = 200,
                   honesty = TRUE,
                   honesty.fraction = 0.5,
                   tune.parameters = "all"
                   )
```

Causal Forests Example

Setting up the causal forest:

- `X`: the covariate matrix
- `Y`: the outcome vector (our measure of student achievement)
- `W`: the treatment vector

```
# Define the random forest
cf ← causal_forest(X = Xmat,
                   Y = Y,
                   W = Z,
                   Y.hat = Y_hat,
                   W.hat = W_hat,
                   clusters = schoolid,
                   equalize.cluster.weights
= TRUE,
                   num.trees = 2000,
                   honesty = TRUE,
                   honesty.fraction = 0.5,
                   tune.parameters = "all"
                   )
```

Causal Forests Example

Setting up the causal forest:

- `X`: the covariate matrix
- `Y`: the outcome vector (our measure of student achievement)
- `W`: the treatment vector
- `Y.hat`: our prediction of Y as a function of X
- `W.hat`: our prediction of W as a function of X

```
# Define the random forest
cf ← causal_forest(X = Xmat,
                   Y = Y,
                   W = Z,
                   Y.hat = Y_hat,
                   W.hat = W_hat,
                   clusters = schoolid,
                   equalize.cluster.weights
= TRUE,
                   num.trees = 200,
                   honesty = TRUE,
                   honesty.fraction = 0.5,
                   tune.parameters = "all"
                   )
```

Causal Forests Example

Setting up the causal forest:

- `clusters`: the cluster identifier vector
- `equalize.cluster.weights`: whether to draw equal sample sizes from each cluster

```
# Define the random forest
cf ← causal_forest(X = Xmat,
                   Y = Y,
                   W = Z,
                   Y.hat = Y_hat,
                   W.hat = W_hat,
                   clusters = schoolid,
                   equalize.cluster.weights
= TRUE,
                   num.trees = 200,
                   honesty = TRUE,
                   honesty.fraction = 0.5,
                   tune.parameters = "all"
                   )
```

Causal Forests Example

Setting up the causal forest:

- `clusters`: the cluster identifier vector
- `equalize.cluster.weights`: whether to draw equal sample sizes from each cluster
- `num.trees`: size of forest to grow (default is 2000)

```
# Define the random forest
cf ← causal_forest(X = Xmat,
                   Y = Y,
                   W = Z,
                   Y.hat = Y_hat,
                   W.hat = W_hat,
                   clusters = schoolid,
                   equalize.cluster.weights
= TRUE,
                   num.trees = 200,
                   honesty = TRUE,
                   honesty.fraction = 0.5,
                   tune.parameters = "all"
                   )
```

Causal Forests Example

Setting up the causal forest:

- `honesty`: whether or not to grow honestly (default to TRUE)
- `honesty.fraction`: the subsample portion used for growing trees (defaults to 50%)

```
# Define the random forest
cf ← causal_forest(X = Xmat,
                   Y = Y,
                   W = Z,
                   Y.hat = Y_hat,
                   W.hat = W_hat,
                   clusters = schoolid,
                   equalize.cluster.weights
= TRUE,
                   num.trees = 200,
                   honesty = TRUE,
                   honesty.fraction = 0.5,
                   tune.parameters = "all"
                   )
```

Causal Forests Example

Setting up the causal forest:

- `tune.parameters`: which/whether to tune parameters
  - `none, all, sample.fraction, mtry, min.node.size, honesty.fraction, honesty.prune.leaves, alpha, imbalance.penalty`

```
# Define the random forest
cf ← causal_forest(X = Xmat,
                   Y = Y,
                   W = Z,
                   Y.hat = Y_hat,
                   W.hat = W_hat,
                   clusters = schoolid,
                   equalize.cluster.weights
= TRUE,
                   num.trees = 200,
                   honesty = TRUE,
                   honesty.fraction = 0.5,
                   tune.parameters = "all"
                   )
```

Variable Importance

Looking at **variable importance** with `variable_importance(forest)`:

- Scaled count of how frequently variables were chosen to split on (i.e. that they maximized treatment effect heterogeneity)

covar	imp
X1	0.167
S3	0.128
X5	0.125
X2	0.116
X4	0.114
X3	0.0953

Split Frequencies

Alternatively, we can obtain the raw split frequencies with...

`split_frequencies(forest, max.depth)`

```
# getting split frequencies 4 branches deep
splitfreq ← split_frequencies(cf, max.depth = 4)
colnames(splitfreq) ← colnames(Xmat)
rownames(splitfreq) ← paste("Depth", 1:4)
splitfreq[,1:8]
```

```
##         S3 C2 C3 X1 X2 X3 X4 X5
## Depth 1 24  3  3 40 24 20 22 26
## Depth 2 50 13 29 29 38 24 50 40
## Depth 3 65 42 34 43 52 44 41 42
## Depth 4 74 61 35 45 35 54 42 59
```

Variable Selection

Our variable importance and split frequencies reveals useful information about the covariates: some **really don't matter**

Since we're only considering a random subset of covariates at each potential split, by including covariates without explanatory power for treatment effect heterogeneity, we're likely **adding noise** and **diluting** the performance of our forest.

Solution: re-run the tree, keeping only the **most important variables**

- Athey and Wager (2019): keep only variables greater than mean importance

Variable Selection

Selecting the "important" covariates:

```
varimp ← variable_importance(cf)
X_sel ← Xmat[,which(varimp> mean(varimp))]
colnames(X_sel)
```

```
## [1] "S3"                "X1"                "X2"                "X3"
## [5] "X4"                "X5"                "factor(nslm$C1)4"
```

Causal Forests Example

And rerunning the forest with just these variables:

```
cf_sel ← causal_forest(X = X_sel,
                       Y = Y,
                       W = Z,
                       num.trees = 2000,
                       honesty = TRUE,
                       honesty.fraction = 0.5,
                       tune.parameters = "all"
                       )
```

Causal Forests Treatment Effects

Use the `average_treatment_effect()` function to get one of several **average treatment effects**:

- `target.sample = "all"`: The ATE
  - $E[Y(1) - Y(0)]$
- `target.sample = "treated"`: The ATT
  - $E[Y(1) - Y(0) \mid W_i = 1]$

Causal Forests Treatment Effects

Use the `average_treatment_effect()` function to get one of several **average treatment effects**:

- `target.sample = "control"`: The average treatment effect on the control
  - $E[Y(1) - Y(0) \mid W_i = 0]$
- `target.sample = "overlap"`: The overlap-weighted average treatment effect on the control
  - $E[e(X)(1 - e(X))(Y(1) - Y(0))]/E[e(X)(1 - e(X))]$
  - Where $e(X) = P[W_i = 1 \mid X_i = x]$

Causal Forests Treatment Effects

Use the `average_treatment_effect()` function to get one of several **average treatment effects**:

```
ate ← average_treatment_effect(cf_sel, target.sample = "all")
ate
```

```
##    estimate     std.err
## 0.25907390 0.01090293
```

Causal Forests Treatment Effects

Retrieving the out-of-bag predictions for $\tau(X)$ with `predict()`:

```
tauhat_oob ← predict(cf_sel, estimate.variance = TRUE)
head(tauhat_oob)
```

predictions	variance.estimates	debiased.error	excess.error
0.248	0.00101	0.0312	1.02e-05
0.202	0.00242	7.43e-05	1.51e-05
0.239	0.00129	0.0219	1.02e-05
0.249	0.0013	0.194	1.01e-05
0.244	0.00118	0.408	9.42e-06
0.246	0.00245	0.122	1.04e-05

Causal Forest Error

In addition to our predicted treatment effects and variance estimate, we get **two types of error**:

Debiased Error

- Estimate of the **"R-loss" Criterion**
  - i.e. the **predictive fit** of our forest

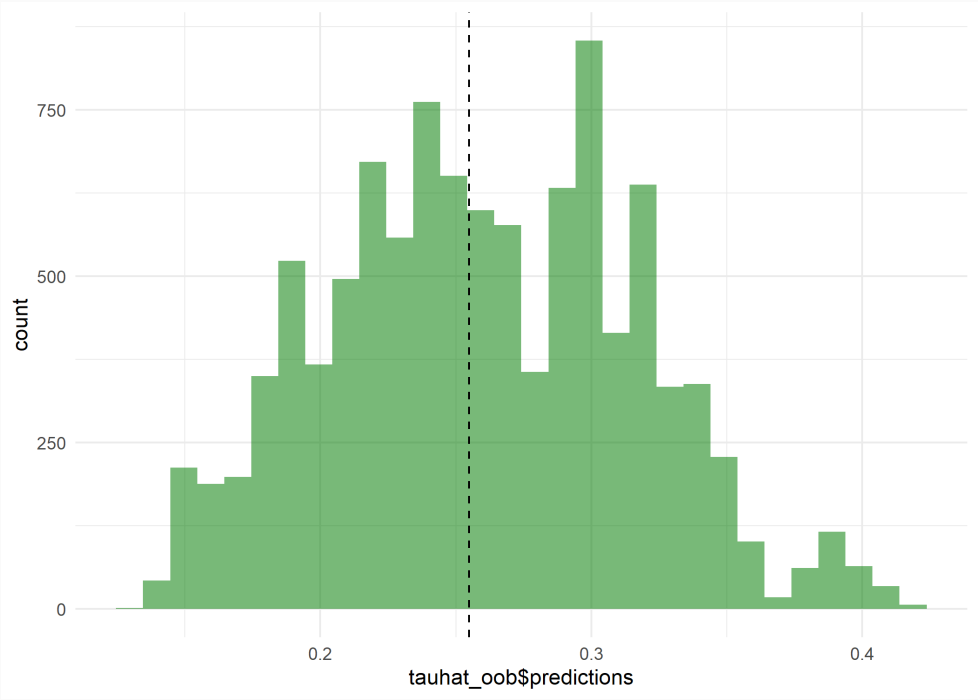
Excess Error

- Error resulting from **random nature of forests**
  - i.e. how **unstable** our estimates would be if we re-grew forests with the same data/setup
  - We want this to be very small relative to estimate's variance

Treatment Effects

Plotting the distribution of individual treatment effects (and relative to the OLS mean estimate:)

```
ggplot() +
  geom_histogram(aes(tauhat_oob$predictions), fill = "forestgreen", alpha = 0.6) +
  geom_vline(aes(xintercept = reg1$coefficients["Z"]), linetype = "dashed") +
  theme_minimal()
```



ATE Subset Comparison

We can also use `subset` to see differences in top/bottom treatment effects

```
# Getting boolean vectors of top/bottom quartile treatment effects
high_effect ← tauhat_oob$predictions > quantile(tauhat_oob$predictions,
0.75)
low_effect ← tauhat_oob$predictions < quantile(tauhat_oob$predictions,
0.25)
```

ATE Subset Comparison

We can also use `subset` to see differences in top/bottom treatment effects

```
# getting ATE for each group
ate_high =
average_treatment_effect(cf,
subset = high_effect)
ate_high
```

```
##   estimate    std.err
## 0.35965572 0.04742362
```

```
# getting ATE for each group
ate_low =
average_treatment_effect(cf,
subset = low_effect)
ate_low
```

```
##   estimate    std.err
## 0.21954032 0.03526803
```

```
## [1] "95% CI for difference in ATE: 0.14 +/- 0.116"
```

Causal Forests Example

Re-estimating the forest **without clusters** shows the importance of accounting for the cluster assignment:

```
cf_noclust ← causal_forest(X = X_sel,
                    Y = Y,
                    Y.hat = predict(Y_forest)$predictions,
                    W = Z,
                    W.hat = predict(W_forest)$predictions,
                    num.trees = 2000,
                    honesty = TRUE,
                    honesty.fraction = 0.5,
                    tune.parameters = "all"
                    )
```

Test Calibration

Looking at the **Test Calibration** results for each forest shows the importance of clustering.

`test_calibration()` Regresses the forest error for prediction of Y on **two terms:**

1. `Mean Prediction`: indication of how well the forest captures the mean ATE
   - Coefficient indistinguishable from 1 $\longrightarrow$ accurate mean prediction
2. `Differential Prediction:` how well the forest captures heterogeneity in the ATE
   - Coefficient distinguishable from 0 $\longrightarrow$ confirms existence of heterogeneous treatment effects
   - Coefficient indistinguishable from 1 $\longrightarrow$ accurately capture heterogeneity

Test Calibration

First for the clustered forest:

```
test_calibration(cf_sel)
```

```
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                               Estimate Std. Error t value    Pr(>t)
## mean.forest.prediction        0.999506   0.042055 23.7669 < 2.2e-16
***
## differential.forest.prediction 1.144052   0.195106  5.8637 2.332e-09
***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Test Calibration

Looking at the **Test Calibration** results for each forest shows the importance of clustering.

Next for the unclustered forest:

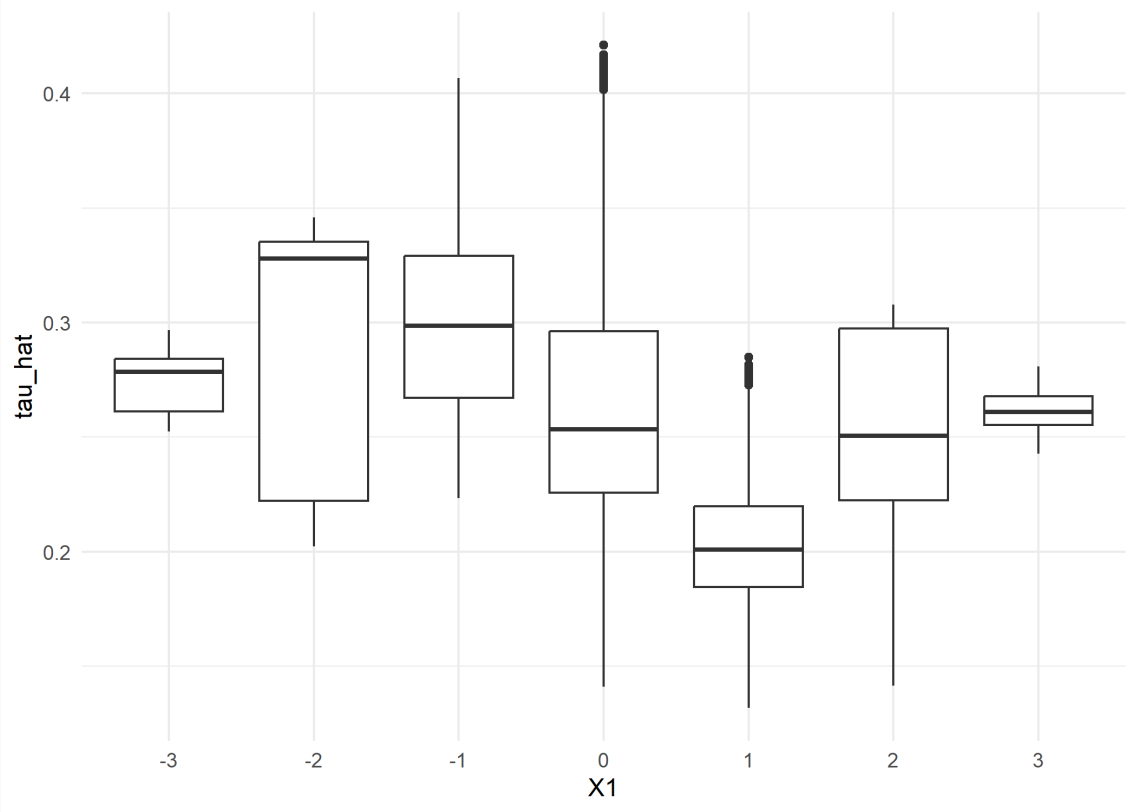
```
test_calibration(cf_noclust)
```

```
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                               Estimate Std. Error t value    Pr(>t)
## mean.forest.prediction         1.01114    0.04496 22.4900 < 2.2e-16
***
## differential.forest.prediction  0.55595    0.11029  5.0409 2.355e-07
***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Treatment Effect Heterogeneity

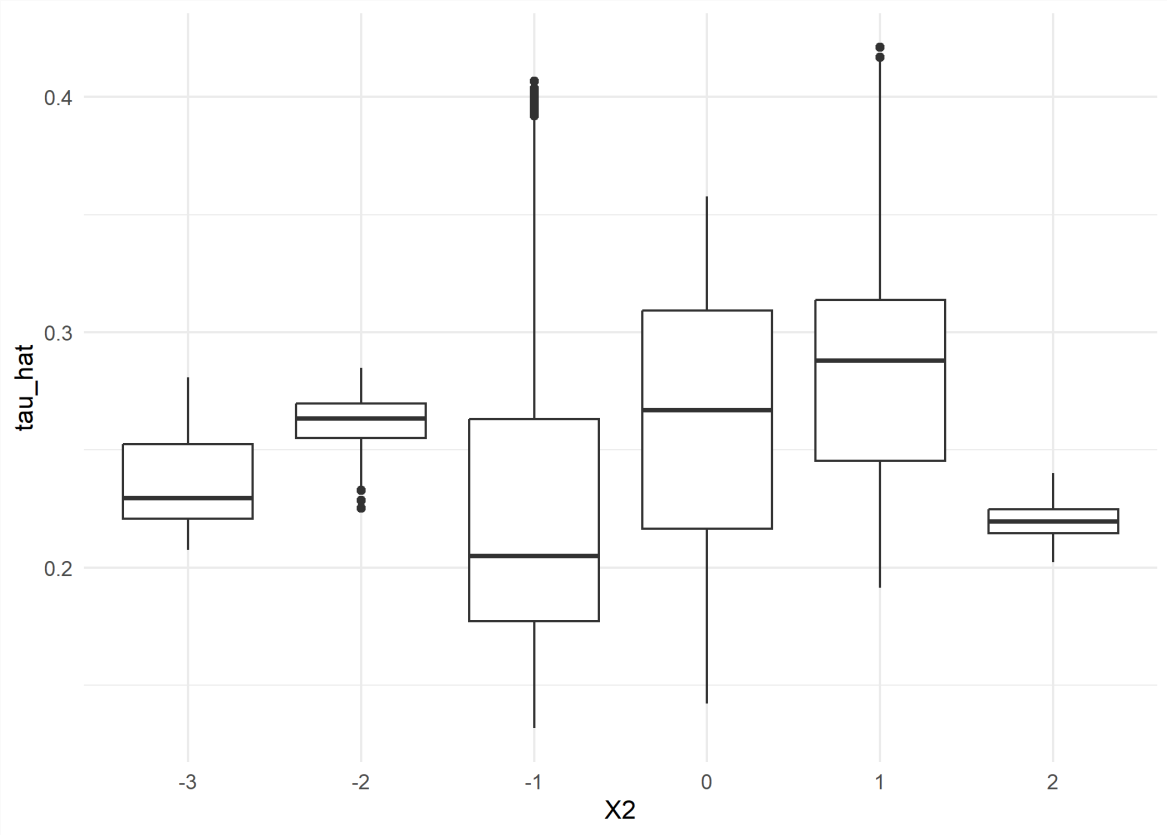
It looks like there's a clear effect of the intervention nudge on achievement, but what about the mediating effects of `X1` and `X2`?

First, let's look at the variation in treatment effects by the values of `X1`:



Treatment Effect Heterogeneity

Repeating for pre-existing achievement levels (`X2`):



It appears that the nudge had the least impact in schools with the highest pre-existing achievement level (`X2`)

Covariate Correlation

One potential limitation: there are a lot of factors at the school-level associated with pre-existing achievement levels.

```
x2_reg ← feols(X2 ~ X1 + X3 + X4 + X5 + C2 + C3  | XC + C1, data = nslm)
etable(x2_reg)
```

	x2_reg
Dependent Var.:	X2
X1	-0.2858*** (0.0080)
X3	-0.2635*** (0.0082)
X4	-0.0902*** (0.0076)
X5	0.2635*** (0.0076)
C2	0.0023 (0.0118)

Simulations

Potential solution: use a new "simulated" covariate matrix to predict treatment effects if schools had different achievement levels, holding other characteristics fixed.

Let's write a function that will let us obtain treatment effect predictions if we swap the value of `X2` with a particular value in all rows:

```
sim_x2 ← function(val){
  # replace all X2 values in the Xmat with the specified value
  X_sim ← mutate(X_sel, X2 = val)

  # predict using the trained forest with the new data
  res ← data.frame(preds = predict(cf_sel, newdata = X_sim)$predictions) %>%
    mutate(X2_val = val)
  return(res)
}
```

Simulations

Next, build a vector of X2 values to predict for:

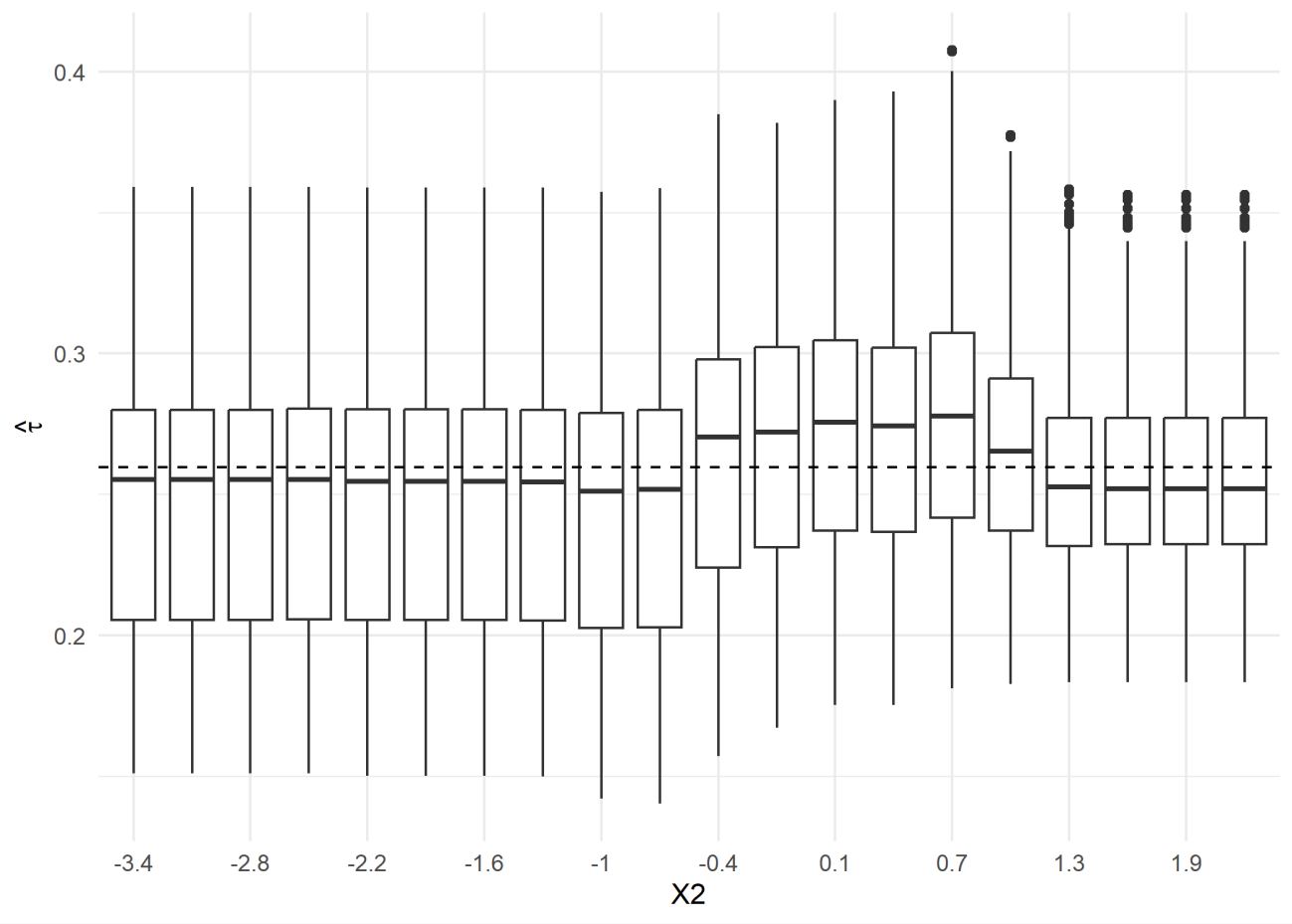
```
# get range of X2 to predict over
min_x2 ← min(Xmat$X2)%>% round(2)
max_x2 ← max(Xmat$X2) %>% round(2)
x2_seq ← seq(min_x2, max_x2, length.out = 20)
```

And running:

```
# Vectorizing and saving out
tic()
sim_x2_df ← map_dfr(x2_seq, sim_x2)
toc()
saveRDS(sim_x2_df, "C:/Users/searsja1/OneDrive - Michigan State
University/Github/AFRE-891-991-SS25/Lecture Slides/11-
ML/data/x2_sim_df.rds")
```

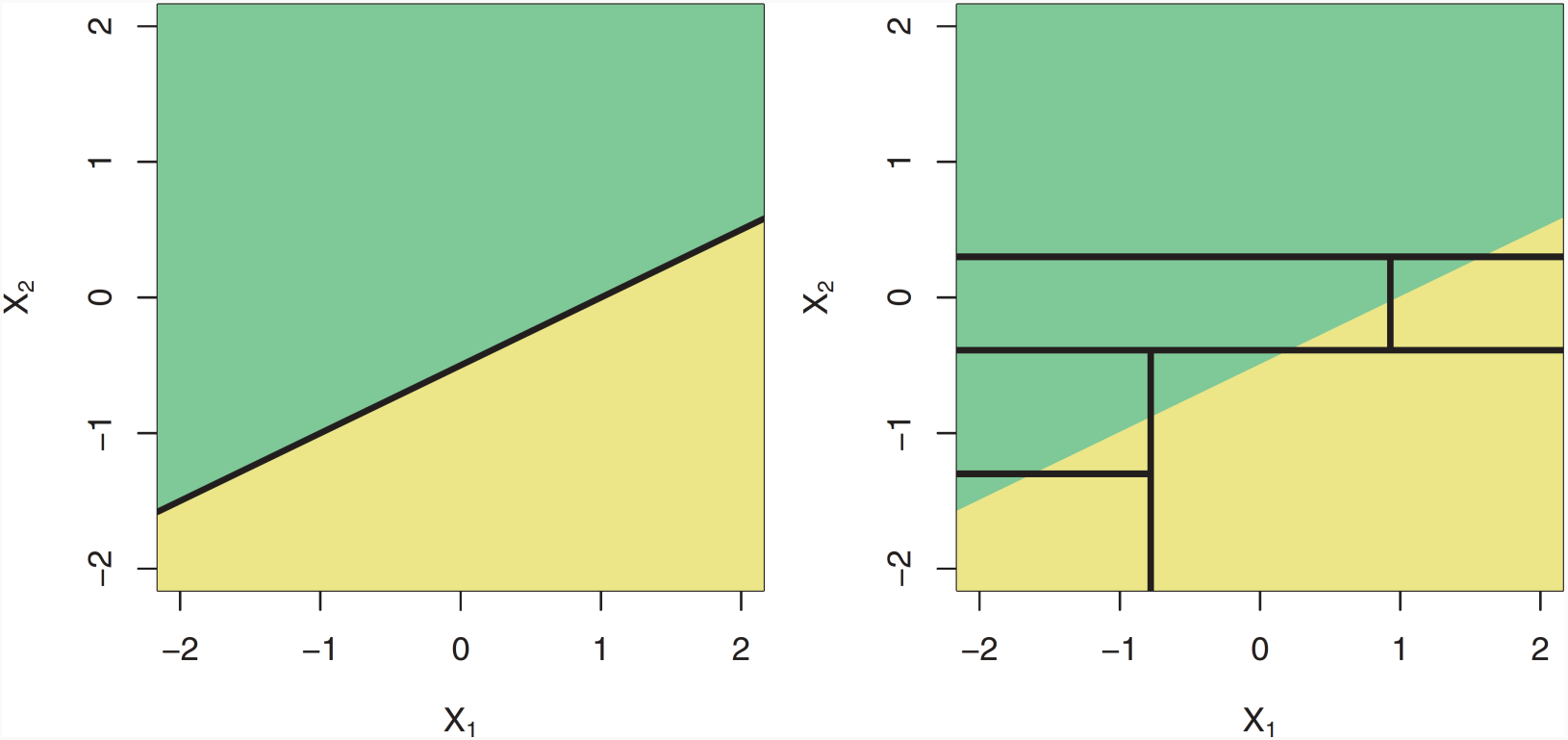
Simulations

Plotting the results suggests that other factors changing play important roles in the treatment effect too (changing `X2` in isolation not sufficient)



Linear Boundaries

What if I want to use these methods to estimate heterogeneous treatment effects but I'm concerned about replicating **linear boundaries**?



A: local linear forests

Local Linear Forests

Developed in Friedberg et al. (2021), local linear forests combine the local smoothness benefits of local linear regression with the high-dimensional flexibility of random forests.

Random Forests

- Highly data-adaptive
- Struggle to capture smoothness in conditional mean function

Local Linear Regression

- Great at capturing locally smooth signals
- Quickly deteriorates due to curse of dimensionality
  - Relies on Euclidean distance, which loses locality even in 4-5 dimensions

Local Linear Forests

Local linear forests function as a Two-stage adaptive kernel method:

1. Grow a random forest with splits chosen to minimize the sum of squared errors between the resulting two branches
    - Uses a standard splitting procedure but splitting on the **residuals from a ridge regression**
    - Partitions the covariate space based on local effects, captures existing linear signals
2. Use the forest to obtain weights $\alpha_i(x_0)$
3. Rather than use the weights to fit the local average at x_0 as in random forests, Use the $\alpha_i(x_0)$ weights to fit a **local linear regression**
    - Includes a ridge penalty for regularization

Local Linear Forests

More formally:

Consider two parameters:

1. $\mu(x_0)$: the local average at x_0
2. $\theta(x_0)$: the slope of the local line
    - Correction for local trend in $X_i - x$

Goal: solve for estimates $\hat{\mu}(x_0), \hat{\theta}(x_0)$ that minimize

$$\sum_{i=1}^{n} \underbrace{\alpha_i(x_0)}_{\text{Weights}}(Y_i - \mu(x_0) - \underbrace{(X_i - x_0)\theta(x_0)}_{\text{Local Trend Correction}})^2 + \underbrace{\lambda||\theta(x_0)||_2^2}_{RidgePenalty}$$

Local Linear Forests

Local linear forests also extend to causal forests:

1. Use local linear forests to obtain the estimates for $\left(\hat{m}^{-1}(X_i),\ \hat{e}^{-1}(X_i)\right)$

   - i.e. the leave-one-out initial predictions for Y and W obtained from auxiliary forests

Local Linear Forests

Local linear forests also extend to causal forests:

2. Estimate conditional average treatment effect $\hat{\tau}$ as the arg min of

$$\sum_{i=1}^{n} \alpha_i(x_0)\Big(Y_i - \hat{m}^{-1}(x_i) - a - (X_i - x_0)\theta_a \\ - (\tau + \theta_t(X_i - x_0))(W_i - \hat{e}^{-1}(X_i))\Big)^2 \\ + \lambda_\tau ||\theta_\tau||_2^2 + \lambda_a ||\theta_a||_2^2$$

- a intercept (0 if $\left(\hat{m}^{-1}(X_i),\ \hat{e}^{-1}(X_i)\right)$ estimates are accurate)
- λ_τ and λ_a: ridge penalties for local trends in treatment and outcome
  - As these get large, we recover a standard causal forest

Local Linear Forests

To estimate a local linear causal forest:

1. Use `ll_regression_forest()` instead of `regression_forest()` to obtain the initial estimates of $\left(\hat{m}_{ll}^{-1}(X_i),\ \hat{e}_{ll}^{-1}(X_i))\right)$

```
# Got LL forest predictions of Y
Y_ll_forest ← ll_regression_forest(X = Xmat, Y = Y,
                                   clusters = schoolid,
                                   equalize.cluster.weights = TRUE,
                                   # make forest splits on ridge residuals
                                   enable.ll.split = TRUE)
Y_ll_hat ← predict(Y_ll_forest,
                   # standardize ridge penalty by covariance
                   ll.weight.penalty = T)$predictions
# Repeat for W
W_ll_forest ← ll_regression_forest(X = Xmat, Y = Z, clusters =
schoolid, equalize.cluster.weights = TRUE, enable.ll.split = TRUE)
W_ll_hat ← predict(W_ll_forest, ll.weight.penalty = T)$predictions
```

Local Linear Forests

To estimate a local linear causal forest:

2. Run Causal forest using the local linear forest estimates from 1.

```
cf_sel_ll ← causal_forest(
  # unchanged parameters:
  X = X_sel, Y = Y, W = Z, clusters = schoolid, equalize.cluster.weights
= TRUE, num.trees = 2000, honesty = TRUE, honesty.fraction = 0.5,
tune.parameters = "all",
  # changed inputs:
                      Y.hat = Y_ll_hat,
                      W.hat = W_ll_hat,
                      )
```

Local Linear Forests

To estimate a local linear causal forest:

3. Use local linear arguments of `predict()` when generating predictions for τ

```
tauhat_ll_oob ← predict(cf_sel_ll,
                        linear.correction.variables = 1:ncol(X_sel),
                        ll.weight.penalty = TRUE
                        )
```

Local Linear Forests

Comparing overall fit (Root MSE) between the two:

```
rmse_cf ← sqrt(sum((cf_sel$Y.orig -
cf_sel$Y.hat)^2)/length(cf_sel$Y.orig))
rmse_ll ← sqrt(sum((cf_sel_ll$Y.orig -
cf_sel_ll$Y.hat)^2)/length(cf_sel_ll$Y.orig))
data.frame(Forest = c("Causal Forest", "Local Linear Causal Forest"),
           RMSE = c(rmse_cf, rmse_ll) %>% round(4))
```

Forest	RMSE
Causal Forest	0.537
Local Linear Causal Forest	0.551

Local Linear Forests

Comparing test calibration results:

```
test_calibration(cf_sel)
```

```
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                               Estimate Std. Error t value    Pr(>t)
## mean.forest.prediction        0.999506   0.042055 23.7669 < 2.2e-16
***
## differential.forest.prediction 1.144052   0.195106  5.8637 2.332e-09
***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Local Linear Forests

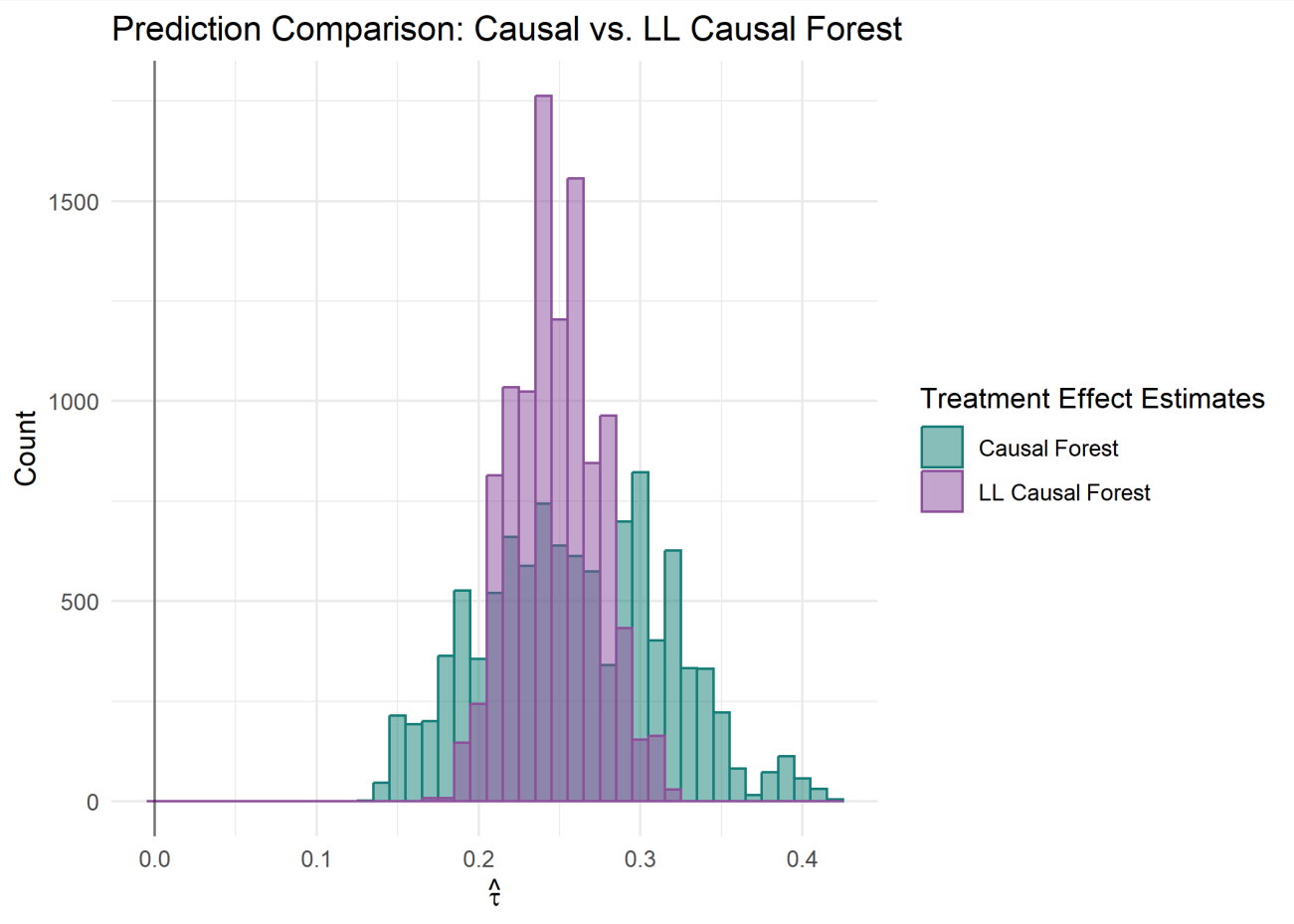
Comparing test calibration results:

```
test_calibration(cf_sel_ll)
```

```
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                                 Estimate Std. Error t value Pr(>t)
## mean.forest.prediction          0.998933   0.049938 20.0035 <2e-16 ***
## differential.forest.prediction -50.880825   5.094837 -9.9867      1
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Local Linear Forests

And comparing predictions from the two approaches:



Dynamics and Causal Forests

The original context for causal forests was in a **cross-sectional setting** with **random treatment assignment**.

- Assumes conditional independence between treatment assignment and outcome

How then can we extend into a typical **panel** setting]?

Proposed Solutions

So far, there have been three main proposed solutions (none currently published)

1. Group Effect

- Jens, Page, and Reeder (2023)
- Estimate the group-level fixed effects, use these for splitting in the forest

2. Difference-in-Differences Causal Forests (DiDCF)

- Gavrilova, Langørgen, and Zoutman (2025)
- Construct base-year differenced outcome $z_{it} = y_{it} - y_{ib}$
- Apply Neyman-orthogonalization
- Estimate CF on differenced outcome for each period $t \neq b$

Proposed Solutions

So far, there have been three main proposed solutions (none currently published)

3. Causal Forests with Fixed Effects (CFFE)

- **Kattenberg, Scheer, and Thiel (2023)**
- Consistent CATT and ATT when common trend and no-anticipation hold at the subgroup level

Manual Re-centering

Note that **manual re-centering** (i.e. demeaning variables before using them in causal forest) *doesn't* solve the temporal issue

- If we demean by time, we use **all units** to identify that effect
- $\Rightarrow$ re-centering isn't local, applies to *every unit*
- $\Rightarrow$ identification fails if treatment effects are **heterogenous**... which is the whole point of using causal forests in the first place!

Group Effect

The "Group Effect" approach of Jens, Page, and Reeder (2023) focuses on overcoming one dimension of limitation: **group-level heterogeneity**.

Idea:

- Create a single variable containing sufficient information on group-level heterogeneity
- Include this variable used to split the causal forest

Required Assumptions:

- Conditional Unconfoundedness: $\{Y_i(1), Y_i(0)\} \perp W_i | X_i, \kappa(C_i)$
  - Potential outcomes are independent of treatment assignment, conditional on covariates and group effects

Group Effect

Define the group effect for group $C_i = c$ as

$$\kappa(c) = E[Y_i - g(X_i, W_i)|C_i = c]$$

Where $g(\cdot)$ is only a function of observables.

Our goal becomes to estimate κ.

- If $g(\cdot)$ is assumed to be linear, then we can use **fixed effects from an OLS regression**
- Alternatively, could use LASSO on higher-order function of observables + group-level indicators (forcing group-level coefficients to be non-zero)
- Or Ridge, Elastic Net, etc.
- Choice depends on your beliefs about $g(\cdot)$

Group Effect

Let's try this with our NLSM example.

1. Build the Group Effect Variable

Assuming linearity of $g(\cdot)$, regress Y on all covariates and group-level fixed effects (here the school level):

```
reg_group ← feols(Y ~ Z | schoolid, data = nslm)
```

And add the fixed effects as a **single variable** into the data:

```
reg_group_fe ← data.frame(schoolid = names(fixef(reg_group)$schoolid)
%>% as.numeric(),
                          group_effect = fixef(reg_group)$schoolid)

nslm ← left_join(nslm, reg_group_fe, by = "schoolid")
```

Group Effect

Now adding the group effect variable into the X matrix:

```
# first select all covariates except for XC and C1
X_ge ← select(nslm, -schoolid:-Y, -XC, - C1)

# Add in the dummies for XC and C1
Xmat_ge ← cbind(X_ge, C1_exp, XC_exp)
```

Group Effect

Re-estimating the causal forest:

```
cf_ge ← causal_forest(
                      X = Xmat_ge,
                      Y = Y,
                      W = Z,
                      equalize.cluster.weights = TRUE,
                      num.trees = 200,
                      honesty = TRUE,
                      honesty.fraction = 0.5,
                      tune.parameters = "all")
```

Group Effect

Checking Importance:

```
varimp_ge ← data.frame(covar =
colnames(Xmat_ge),
                       imp =
variable_importance(cf_ge)
                       ) %>%
  arrange(desc(imp))

head(varimp_ge)
```

covar	imp
group_effect	0.186
X1	0.153
X2	0.135

Compared to the original forest:

```
head(varimp_df)
```

covar	imp
X1	0.167
S3	0.128
X5	0.125
X2	0.116
X4	0.114
X3	0.0953

Table of Contents

Part 3: Tree-Based Methods

1. **Decision Trees**
2. **Random Forests**
3. **Machine Learning for Causal Treatment Effect Estimation**
4. **Deep Learning (if time - not a tree method)**